

Machine Learning

Sara Cristina Basco

January 11, 2026

Contents

1	Introduzione al Machine Learning	4
1.1	Definizione di Machine Learning	4
1.2	Fattori abilitanti del Machine Learning	4
1.3	Applicazioni del Machine Learning	4
1.4	Cos'è un problema di apprendimento	5
1.5	Funzione ideale	5
1.6	Funzione valore	5
1.6.1	Rappresentazione approssimata della funzione valore	6
1.7	Algoritmo di apprendimento	6
1.8	Funzione obiettivo e rappresentazione	7
1.9	Dataset e generalizzazione	7
1.10	Tipologie di problemi di Machine Learning	7
1.10.1	Supervised Learning	7
1.10.2	Unsupervised Learning	7
1.10.3	Reinforcement Learning	8
1.11	Ipotesi e spazio delle ipotesi	8
1.12	Consistenza e Version Space	8
1.13	Algoritmo List-Then-Eliminate	9
1.13.1	Osservazioni	9
1.14	Rappresentazione vs Generalizzazione	10
1.15	Dati rumorosi	10
2	Performance Evaluation	12
2.1	Introduzione alla valutazione delle prestazioni	12
2.2	Errore vero ed errore campionario	12
2.2.1	Errore vero	13
2.2.2	Errore campionario	13
2.3	Intervalli di confidenza	14
2.4	Stima non distorta e trade-off training/testing	14
2.5	k-Fold Cross Validation	14
2.5.1	Descrizione dell'algoritmo	14
2.6	Overfitting	15
2.7	Metriche di valutazione	16
2.7.1	Limiti dell'accuratezza	16
2.7.2	Precision, Recall e F1-score	16
2.7.3	Metriche derivate e curva ROC	17
2.7.4	Dipendenza dal dominio applicativo	17
2.7.5	Matrice di confusione multi-classe	17

2.8	Metriche di valutazione per la regressione	18
2.8.1	Metriche percentuali e coefficiente di determinazione	20
2.8.2	Cross-validation per la regressione	20
3	Decision Tree	22
3.1	Definizione	22
3.1.1	Struttura del Decision Tree	22
3.1.2	Caratteristiche principali	23
3.2	Regole di decisione	24
3.2.1	Decision Tree come insieme di regole	25
3.2.2	Conversione di un decision tree in regole	25
3.3	Algoritmo ID3	26
3.3.1	Costruzione del decision tree	26
3.3.2	Proprietà dell'algoritmo ID3	27
3.3.3	ID3 e regole di decisione	27
3.4	Entropia e Information Gain	27
3.4.1	Entropia	28
3.4.2	Information Gain	28
3.4.3	Schema operativo	29
3.5	Overfitting nei Decision Tree	29
3.6	Pruning	30
3.6.1	Pre-pruning	30
3.6.2	Post-pruning	31
3.7	Random Forest	31
3.7.1	Aggregazione delle predizioni	32
3.7.2	Riduzione dell'overfitting	32
4	Bayesian Learning	36
4.1	Motivazione	36
4.2	Classificazione come stima probabilistica	36
4.3	ML e MAP	37
4.3.1	Maximum Likelihood Classification	37
4.3.2	Maximum A Posteriori Classification	37
4.4	Bayes Optimal Classifier	38
4.5	Naive Bayes Classifier	39
4.5.1	Definizione	39
4.5.2	Formulazione formale	40
4.5.3	Stima delle probabilità	41
5	Probabilistic Models for Classification	45
5.1	Modelli probabilistici generativi	45
5.1.1	Caso gaussiano con covarianza condivisa	45
5.1.2	Stima dei parametri tramite Maximum Likelihood	46
5.1.3	Estensione a K classi	46
5.1.4	Gaussian Naive Bayes	46
5.2	Modelli probabilistici discriminativi	46
5.2.1	Logistic Regression	47
5.2.2	Regressione logistica multi-classe	47
5.2.3	Generalizzazione e spazio delle feature	47

6	Modelli lineari per la classificazione	50
6.1	Dati linearmente separabili	50
6.1.1	Dati non linearmente separabili	51
6.2	Linear Discriminant Function	51
6.2.1	Regola di decisione	51
6.3	Learning di funzioni discriminanti lineari	53
6.4	Least Squares per la classificazione	53
6.4.1	Definizione del problema	53
6.4.2	Funzione di costo	54
6.4.3	Soluzione ottima	54
6.4.4	Regola di classificazione	55
6.4.5	Sensibilità agli outlier	55
6.5	Perceptron	56
6.5.1	Definizione del problema	56
6.5.2	Modello del Perceptron	56
6.5.3	Perceptron Learning Rule	57
6.5.4	Algoritmo del Perceptron	58
6.5.5	Interpretazione neurale	60

Chapter 1

Introduzione al Machine Learning

1.1 Definizione di Machine Learning

Il *Machine Learning* (ML) può essere definito come il campo di studio che si occupa di programmare calcolatori affinché migliorino la loro performance su un determinato compito attraverso l'esperienza. Formalmente, un sistema di apprendimento migliora la propria prestazione rispetto a un compito T , misurata da una metrica di performance P , sulla base dell'esperienza E .

$$\text{Learning} \equiv \text{Improve}(T, P, E)$$

Il Machine Learning è particolarmente utile quando:

- non esiste una competenza umana esplicita;
- gli esseri umani non sono in grado di formalizzare la propria esperienza;
- la soluzione deve adattarsi a casi specifici o a dati in continua evoluzione.

1.2 Fattori abilitanti del Machine Learning

Lo sviluppo del Machine Learning è stato reso possibile dalla convergenza di tre fattori principali:

- progressi teorici e algoritmici;
- disponibilità massiva di dati (Big Data);
- aumento della potenza computazionale, in particolare grazie alle GPU.

1.3 Applicazioni del Machine Learning

Il Machine Learning trova applicazione in numerosi domini, tra cui:

- computer vision;
- riconoscimento vocale e interazione parlata;
- analisi di testi e documenti;

- data analytics;
- controllo robotico;
- giochi e sistemi decisionali.

1.4 Cos'è un problema di apprendimento

Un problema di apprendimento consiste nel migliorare una prestazione nel tempo grazie all'esperienza. Un classico esempio è l'apprendimento del gioco della dama:

- Compito (T): giocare a dama;
- Performance (P): percentuale di partite vinte;
- Esperienza (E): partite giocate contro sé stessi.

La progettazione di un sistema di apprendimento richiede di rispondere alle seguenti domande:

- quale esperienza utilizzare;
- cosa deve essere appreso;
- come rappresentare la conoscenza;
- quale algoritmo usare per apprendere.

1.5 Funzione ideale

In un problema di apprendimento si assume l'esistenza di una funzione ideale che descrive perfettamente il comportamento desiderato. Tale funzione è concettualmente corretta, ma non è disponibile in forma esplicita e non è computabile nella pratica.

Nel caso generale, si tratta di una funzione:

$$f : X \rightarrow Y$$

che associa a ogni possibile istanza di input $x \in X$ l'output corretto $y \in Y$. L'obiettivo del Machine Learning è apprendere una funzione che approssimi il più possibile questa funzione ideale, utilizzando un insieme finito di esempi.

1.6 Funzione valore

Nel problema dell'apprendimento del gioco della dama, la funzione ideale può essere espressa tramite una funzione valore:

$$V(b)$$

che associa a ogni configurazione della scacchiera b un valore reale che rappresenta la qualità della posizione.

Una possibile definizione concettuale della funzione valore è la seguente:

- se b è uno stato finale vinto, allora $V(b) = 100$;
- se b è uno stato finale perso, allora $V(b) = -100$;
- se b è uno stato finale di pareggio, allora $V(b) = 0$;
- se b non è uno stato finale, allora $V(b)$ è il valore del miglior stato finale raggiungibile giocando in modo ottimale.

Questa funzione è corretta dal punto di vista teorico, ma non è computabile, poiché richiederebbe di esplorare completamente lo spazio degli stati e delle possibili partite.

1.6.1 Rappresentazione approssimata della funzione valore

Poiché la funzione valore ideale $V(b)$ non è computabile, si introduce una funzione approssimata $\hat{V}(b)$, appartenente a una classe di funzioni più semplici e computabili.

Nel caso considerato, $\hat{V}(b)$ è rappresentata come una combinazione lineare di feature estratte dalla configurazione della scacchiera:

$$\hat{V}(b) = w_0 + \sum_{i=1}^n w_i f_i(b)$$

dove:

- $f_i(b)$ sono feature note e computabili;
- w_i sono coefficienti reali ignoti;
- $\hat{V}(b)$ approssima il valore reale della configurazione.

Nel caso specifico della dama, le feature includono il numero di pedine, dame e pezzi minacciati per ciascun giocatore. L'apprendimento si riduce quindi alla stima dei pesi w_i .

1.7 Algoritmo di apprendimento

L'apprendimento dei pesi w_i avviene tramite un algoritmo iterativo basato sulla riduzione dell'errore tra il valore stimato e il valore di training.

Nel caso considerato viene utilizzato l'algoritmo *LMS* (*Least Mean Squares*).

Dato un esempio di training b , l'errore è definito come:

$$\text{error}(b) = V_{\text{train}}(b) - \hat{V}(b)$$

I pesi vengono aggiornati secondo la regola:

$$w_i \leftarrow w_i + c \cdot f_i(b) \cdot \text{error}(b)$$

dove c è il learning rate, che controlla la velocità di apprendimento.

L'algoritmo LMS è un metodo di apprendimento incrementale basato sulla discesa del gradiente stocastica.

1.8 Funzione obiettivo e rappresentazione

Nel Machine Learning si assume l'esistenza di una funzione obiettivo (o funzione target) sconosciuta:

$$f : X \rightarrow Y$$

L'obiettivo dell'apprendimento è calcolare una funzione approssimata:

$$\hat{f}(x) \approx f(x)$$

che generalizzi correttamente anche su esempi non presenti nel dataset di training.

1.9 Dataset e generalizzazione

Dato un dataset di addestramento:

$$D = \{(x_i, y_i)\}_{i=1}^n$$

il sottoinsieme degli input osservati è:

$$X_D \subset X \quad \text{con} \quad |X_D| \ll |X|$$

Il vero obiettivo dell'apprendimento non è la correttezza su D , ma la capacità di generalizzare a nuovi campioni $x \notin X_D$.

1.10 Tipologie di problemi di Machine Learning

In base al tipo di dataset e di output, si distinguono tre principali categorie:

1.10.1 Supervised Learning

Il dataset contiene coppie input-output:

$$D = \{(x_i, y_i)\}_{i=1}^n$$

Include:

- **Classificazione:** $Y = \{C_1, \dots, C_k\}$;
- **Regressione:** $Y \subset \mathbb{R}^k$.

Caso particolare: **Concept Learning**, con output binario $Y = \{0, 1\}$.

1.10.2 Unsupervised Learning

Il dataset contiene solo input:

$$D = \{x_i\}_{i=1}^n$$

L'obiettivo è individuare strutture latenti nei dati, come nel clustering o nella riduzione dimensionale.

1.10.3 Reinforcement Learning

Il sistema apprende una politica:

$$\pi : S \rightarrow A$$

sulla base di sequenze di stati, azioni e ricompense:

$$D = \{(s_0, a_1, r_1, s_1, \dots)\}$$

Non è disponibile un output supervisionato diretto, ma solo ricompense sparse e ritardate.

1.11 Ipotesi e spazio delle ipotesi

Un'ipotesi h è una possibile approssimazione della funzione target ideale. Ogni ipotesi rappresenta una funzione che associa a ciascuna istanza di input un valore di output.

Lo **spazio delle ipotesi** H è l'insieme di tutte le ipotesi che il sistema di apprendimento è in grado di rappresentare, dato un certo formalismo o modello scelto.

L'apprendimento può essere visto come un problema di ricerca nello spazio delle ipotesi, in cui si vuole individuare l'ipotesi che massimizza una misura di performance sul dataset di training:

$$h^* = \arg \max_{h \in H} \text{Performance}(h, D)$$

La scelta dello spazio delle ipotesi è cruciale, poiché determina:

- il potere rappresentativo del modello;
- la capacità di generalizzazione;
- la complessità computazionale del processo di apprendimento.

Uno spazio delle ipotesi troppo limitato può non contenere una buona approssimazione della funzione target, mentre uno spazio troppo ampio può portare a scarsa generalizzazione.

1.12 Consistenza e Version Space

Un'ipotesi h si dice **consistente** con un dataset di training D se fornisce la risposta corretta per tutti gli esempi presenti nel dataset:

$$\forall (x, c(x)) \in D \quad h(x) = c(x)$$

Il **Version Space** è definito come il sottoinsieme dello spazio delle ipotesi contenente tutte le ipotesi consistenti con i dati osservati:

$$VS_{H,D} = \{h \in H \mid h \text{ è consistente con } D\}$$

Il Version Space rappresenta l'insieme di tutte le spiegazioni compatibili con i dati disponibili. L'obiettivo di un sistema di apprendimento non è semplicemente trovare una qualunque ipotesi consistente, ma selezionare un'ipotesi che generalizzi correttamente anche su istanze non presenti nel dataset.

Schema concettuale



1.13 Algoritmo List-Then-Eliminate

Un modo concettualmente semplice per calcolare il Version Space è l'algoritmo *List-Then-Eliminate*. Questo algoritmo assume che lo spazio delle ipotesi H sia finito ed enumerabile.

L'algoritmo opera come segue:

1. inizializza il Version Space contenendo tutte le ipotesi in H ;
2. per ogni esempio di training $(x, c(x))$, elimina dal Version Space tutte le ipotesi h tali che $h(x) \neq c(x)$;
3. restituisce l'insieme delle ipotesi rimanenti nel Version Space.

Formalmente:

$$VS_{H,D} = \{h \in H \mid \forall (x, c(x)) \in D, h(x) = c(x)\}$$

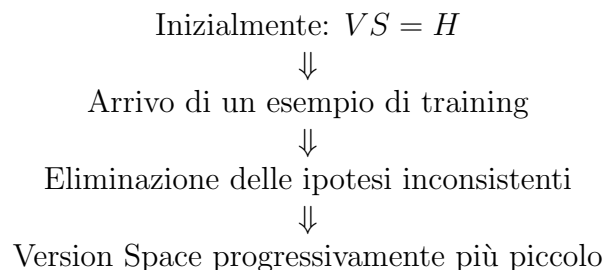
1.13.1 Osservazioni

L'algoritmo List-Then-Eliminate è corretto dal punto di vista teorico, ma non è utilizzabile nella pratica, poiché richiede l'enumerazione esplicita di tutte le ipotesi in H .

Il suo ruolo principale è quello di chiarire:

- il concetto di consistenza;
- il significato operativo di Version Space;
- il legame tra dati di training e riduzione dello spazio delle ipotesi.

Schema concettuale



1.14 Rappresentazione vs Generalizzazione

Esiste un compromesso fondamentale tra:

- **potere rappresentativo** dello spazio delle ipotesi;
- **potere di generalizzazione**.

Se lo spazio delle ipotesi è troppo potente (può rappresentare ogni sottoinsieme di X), il sistema perde capacità di generalizzare.

1.15 Dati rumorosi

Nei problemi reali, il dataset può contenere rumore:

$$y_i \neq c(x_i)$$

In questi casi:

- il Version Space può essere vuoto;
- non esistono ipotesi perfettamente consistenti;
- è necessario adottare metodi statistici robusti.

RIEPILOGO

Machine Learning è un approccio *data-driven* che si basa sull'uso dei dati per costruire modelli quando una descrizione completa dell'ambiente non è disponibile. L'apprendimento è definito come il miglioramento delle prestazioni su un compito, sulla base dell'esperienza e misurato tramite una metrica di performance.

Nel Machine Learning si assume l'esistenza di una funzione ideale (target) sconosciuta, che viene approssimata mediante una funzione computabile. Nel caso dell'apprendimento del gioco della dama, tale funzione è una funzione valore $V(b)$, che assegna a ogni configurazione un valore numerico. Poiché $V(b)$ non è computabile, si introduce una funzione approssimata $\hat{V}(b)$, rappresentata come combinazione lineare di feature, i cui parametri vengono appresi dai dati.

L'apprendimento dei parametri avviene tramite un algoritmo iterativo, come l'algoritmo LMS, che aggiorna i pesi riducendo l'errore tra il valore stimato e il valore di training.

Tipologie di problemi di Machine Learning:

- **Supervised Learning:** il dataset contiene coppie input-output; include problemi di regressione e classificazione;
- **Unsupervised Learning:** il dataset contiene solo input, senza etichette; l'obiettivo è scoprire strutture latenti nei dati;
- **Reinforcement Learning:** l'agente apprende una politica sulla base di ricompense ritardate e interazioni con l'ambiente.

Notazione ricorrente:

- X : spazio delle istanze;
- $x \in X$: una singola istanza;
- $D = \{(x_i, c(x_i))\}_{i=1}^n$: dataset di training;
- c : funzione target ideale;
- H : spazio delle ipotesi;
- $h \in H$: ipotesi;
- $h(x)$: valore stimato dall'ipotesi su x .

Un'ipotesi è detta **consistente** con il dataset se fornisce la risposta corretta per tutti gli esempi di training. Il **Version Space** è l'insieme di tutte le ipotesi consistenti con i dati osservati e rappresenta lo spazio delle soluzioni compatibili con il dataset. La scelta dello spazio delle ipotesi è cruciale: uno spazio troppo semplice limita il potere rappresentativo, mentre uno spazio troppo complesso riduce la capacità di generalizzazione. Trovare il giusto equilibrio è fondamentale per evitare fenomeni di *underfitting* e *overfitting*.

Chapter 2

Performance Evaluation

2.1 Introduzione alla valutazione delle prestazioni

La valutazione delle prestazioni è il processo mediante il quale si stima la qualità di una ipotesi o di un algoritmo di apprendimento nel generalizzare a nuovi dati non osservati.

Dato che un modello di Machine Learning viene addestrato su un insieme finito di esempi, non è possibile osservare direttamente il suo comportamento sull'intera distribuzione dei dati. La valutazione delle prestazioni fornisce quindi una **stima statistica** dell'accuratezza del modello su istanze future.

In termini generali, la valutazione delle prestazioni risponde a domande del tipo:

- quanto è accurata l'ipotesi appresa?
- quanto è affidabile questa stima?
- come si confrontano due ipotesi o due algoritmi diversi?

È importante sottolineare che la valutazione delle prestazioni non restituisce un valore assoluto, ma una stima soggetta a incertezza, che dipende:

- dalla quantità di dati disponibili;
- dalla modalità di campionamento;
- dalla metrica di valutazione utilizzata.

La difficoltà principale nella valutazione delle prestazioni risiede nel fatto che l'errore vero di un modello non è osservabile direttamente. Per questo motivo si introducono stime basate su campioni finiti di dati, come l'errore campionario.

2.2 Errore vero ed errore campionario

Sia $f : X \rightarrow Y$ la funzione target ideale e sia D una distribuzione di probabilità sugli input. Dato un modello appreso h , l'obiettivo della valutazione delle prestazioni è stimare quanto spesso h fornisca una predizione errata su esempi futuri.

2.2.1 Errore vero

L'**errore vero** di un'ipotesi h è definito come la probabilità che h commetta un errore su un'istanza estratta casualmente secondo la distribuzione D :

$$error_D(h) = \Pr_{x \sim D}[f(x) \neq h(x)]$$

L'errore vero rappresenta la reale capacità di generalizzazione del modello. Tuttavia, non può essere calcolato direttamente, poiché la distribuzione D e la funzione target f sono sconosciute.

2.2.2 Errore campionario

Poiché l'errore vero non è osservabile, si introduce l'**errore campionario**, calcolato su un insieme finito di esempi S :

$$error_S(h) = \frac{1}{|S|} \sum_{x \in S} \delta(f(x) \neq h(x))$$

L'errore campionario è una quantità misurabile e rappresenta una stima dell'errore vero. Tuttavia, essendo calcolato su un campione finito, dipende dagli esempi contenuti in S .

Esempio

Si consideri un problema di classificazione binaria e un modello h . Supponiamo di testare il modello su un insieme di 20 esempi e osservare 2 errori. In questo caso:

$$error_S(h) = \frac{2}{20} = 0.1$$

Questo valore rappresenta l'errore campionario. Tuttavia, se il test fosse stato effettuato su un diverso campione di 20 esempi estratti dalla stessa distribuzione, il numero di errori osservati avrebbe potuto essere diverso.

L'errore vero $error_D(h)$ rappresenta invece la frazione di errori che il modello commetterebbe su un numero infinito di esempi estratti dalla distribuzione D . L'errore campionario è quindi una realizzazione casuale di una variabile aleatoria che approssima l'errore vero.

Interpretazione statistica

L'errore campionario $error_S(h)$ è una variabile casuale, poiché dipende dal campione S estratto dalla distribuzione D . Campioni diversi producono stime diverse dello stesso errore vero.

Il problema centrale della valutazione delle prestazioni consiste nel determinare:

- quanto bene $error_S(h)$ approssimi $error_D(h)$;
- quanto è affidabile questa approssimazione.

2.3 Intervalli di confidenza

Se il campione di test contiene almeno 30 esempi ed è indipendente dall'ipotesi, con probabilità approssimativamente pari a $N\%$, l'errore vero soddisfa:

$$error_D(h) \in error_S(h) \pm z_N \sqrt{\frac{error_S(h)(1 - error_S(h))}{n}}$$

dove z_N è un coefficiente dipendente dal livello di confidenza desiderato.

2.4 Stima non distorta e trade-off training/testing

Per ottenere una stima non distorta, i dati utilizzati per la valutazione devono essere indipendenti da quelli usati per l'addestramento. Una suddivisione tipica è:

- $\frac{2}{3}$ dei dati per il training;
- $\frac{1}{3}$ dei dati per il testing.

Esiste un trade-off: più dati di training migliorano il modello, ma peggiorano la stima; più dati di test migliorano la stima, ma peggiorano il modello.

2.5 k-Fold Cross Validation

La suddivisione dei dati in un singolo training set e un singolo test set fornisce una stima dell'errore vero, ma tale stima può avere elevata varianza, poiché dipende fortemente dalla particolare partizione dei dati.

La **k-Fold Cross Validation** è una tecnica che consente di ridurre questa variabilità, utilizzando in modo più efficiente l'intero dataset disponibile.

2.5.1 Descrizione dell'algoritmo

Dato un dataset D , esso viene suddiviso in k sottoinsiemi disgiunti:

$$D = S_1 \cup S_2 \cup \dots \cup S_k$$

Per ogni iterazione $i = 1, \dots, k$:

- il sottoinsieme S_i viene utilizzato come test set;
- i rimanenti $k - 1$ sottoinsiemi costituiscono il training set $T_i = D \setminus S_i$;
- viene addestrato un modello $h_i = L(T_i)$;
- viene calcolato l'errore $error_{S_i}(h_i)$.

L'errore finale dell'algoritmo di apprendimento L è stimato come:

$$error_{L,D} = \frac{1}{k} \sum_{i=1}^k error_{S_i}(h_i)$$

Interpretazione

La k-Fold Cross Validation fornisce una stima dell'**valore atteso** dell'errore campionario di un algoritmo di apprendimento.

Ogni esempio del dataset viene utilizzato:

- una sola volta come dato di test;
- $k - 1$ volte come dato di training.

In questo modo, la stima dell'errore:

- è meno dipendente da una singola partizione dei dati;
- presenta una varianza inferiore rispetto a un singolo split.

Esempio

Si consideri un dataset composto da 100 esempi e una 5-Fold Cross Validation. Il dataset viene suddiviso in 5 sottoinsiemi da 20 esempi ciascuno.

Ad ogni iterazione:

- 80 esempi vengono utilizzati per il training;
- 20 esempi vengono utilizzati per il testing.

Al termine delle 5 iterazioni, ciascun esempio è stato utilizzato esattamente una volta come test set. L'errore finale è la media degli errori osservati nelle singole iterazioni.

Schema concettuale



2.6 Overfitting

Nel Machine Learning l'obiettivo non è minimizzare l'errore sui dati di training, ma ottenere buone prestazioni su dati futuri non osservati. Il fenomeno dell'**overfitting** si verifica quando un modello si adatta eccessivamente ai dati di training, perdendo capacità di generalizzazione.

Formalmente, data una ipotesi $h \in H$, si dice che h presenta overfitting rispetto a un dataset S se esiste un'altra ipotesi $h' \in H$ tale che:

$$error_S(h) < error_S(h') \quad \text{e} \quad error_D(h) > error_D(h')$$

In altre parole, h è migliore sui dati di training, ma peggiore sui dati futuri rispetto a h' .

L'overfitting è una conseguenza di uno spazio delle ipotesi troppo complesso rispetto alla quantità di dati disponibili. Un modello con elevato potere rappresentativo può adattarsi anche al rumore presente nel dataset, trattandolo come se fosse informazione rilevante.

Esempio

Si consideri un problema di classificazione e due modelli h e h' . Il modello h è molto complesso, mentre h' è più semplice.

Supponiamo di osservare i seguenti risultati:

$$\begin{aligned} error_S(h) &= 0.02 & error_S(h') &= 0.08 \\ error_D(h) &= 0.25 & error_D(h') &= 0.10 \end{aligned}$$

Il modello h commette pochissimi errori sui dati di training, ma generalizza molto male. Al contrario, il modello h' ha un errore di training più alto, ma un errore vero più basso.

In questo caso, il modello h presenta overfitting, mentre h' rappresenta una soluzione preferibile.

L'overfitting non può essere individuato osservando esclusivamente l'errore di training. È necessario stimare l'errore vero mediante tecniche di valutazione come il test set indipendente o la k-Fold Cross Validation.

2.7 Metriche di valutazione

2.7.1 Limiti dell'accuratezza

Nel caso di problemi di classificazione, una metrica comunemente utilizzata è l'accuratezza:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Tuttavia, l'accuratezza può risultare fuorviante in presenza di dataset sbilanciati, ovvero quando una classe è molto più frequente delle altre. In questi casi, un classificatore banale che predice sempre la classe più frequente può ottenere un'alta accuratezza pur non essendo utile.

2.7.2 Precision, Recall e F1-score

Per valutare meglio le prestazioni in presenza di dataset sbilanciati, si introducono metriche alternative basate sulla matrice di confusione.

La **Recall** (o True Positive Rate) misura la capacità del modello di individuare correttamente i casi positivi:

$$Recall = \frac{TP}{TP + FN}$$

La **Precision** misura la frazione di predizioni positive corrette:

$$Precision = \frac{TP}{TP + FP}$$

La **F1-score** combina precision e recall in un'unica metrica:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

L'importanza relativa di precision e recall dipende dal dominio applicativo.

2.7.3 Metriche derivate e curva ROC

Altre metriche utili sono:

- **True Negative Rate (Specificity):**

$$TNR = \frac{TN}{TN + FP}$$

- **False Positive Rate:**

$$FPR = \frac{FP}{TN + FP}$$

- **False Negative Rate:**

$$FNR = \frac{FN}{TP + FN}$$

La **ROC curve** rappresenta graficamente il compromesso tra True Positive Rate e False Positive Rate al variare della soglia di classificazione. L'**AUC** (Area Under the Curve) fornisce una misura sintetica delle prestazioni complessive del classificatore.

2.7.4 Dipendenza dal dominio applicativo

L'impatto degli errori dipende fortemente dall'applicazione. Ad esempio:

- in ambito medico, un falso negativo può avere conseguenze gravi;
- nella guida autonoma, un falso positivo può causare frenate inutili;
- in sistemi di sicurezza, un falso negativo può compromettere l'intero sistema.

La scelta della metrica di valutazione deve quindi essere guidata dal contesto applicativo e non basarsi esclusivamente sull'accuratezza.

2.7.5 Matrice di confusione multi-classe

Nei problemi di classificazione multi-classe, la matrice di confusione riporta quante volte un'istanza della classe C_i viene classificata come appartenente alla classe C_j .

La diagonale principale contiene le classificazioni corrette per ciascuna classe, mentre gli elementi fuori diagonale indicano le classi che vengono più frequentemente confuse. L'analisi della matrice di confusione consente di individuare debolezze specifiche del classificatore.

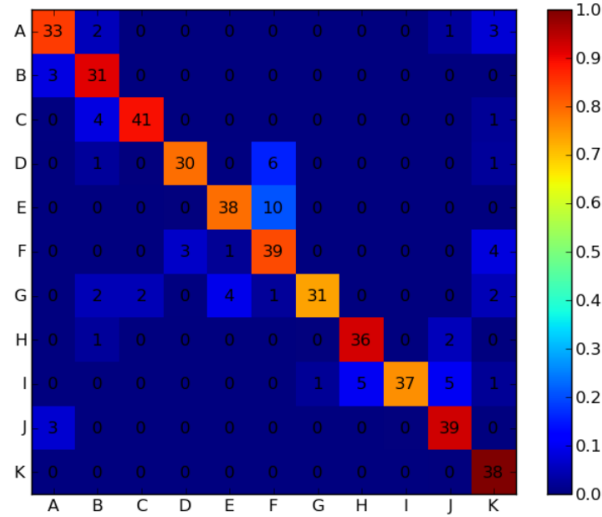


Figure 2.1: Esempio di matrice di confusione per un problema di classificazione. Gli elementi sulla diagonale rappresentano le predizioni corrette.

2.8 Metriche di valutazione per la regressione

Nel caso di problemi di regressione, la valutazione delle prestazioni si basa sulla differenza numerica tra il valore predetto $\hat{f}(x_i)$ e il valore reale t_i .

Le metriche più comuni sono:

- **Mean Absolute Error (MAE)**: misura l'errore medio assoluto tra i valori predetti e i valori reali

$$MAE = \frac{1}{n} \sum_{i=1}^n \left| \hat{f}(x_i) - t_i \right|$$

Il MAE assegna lo stesso peso a tutti gli errori, indipendentemente dalla loro ampiezza. Ogni errore contribuisce in modo lineare alla metrica.

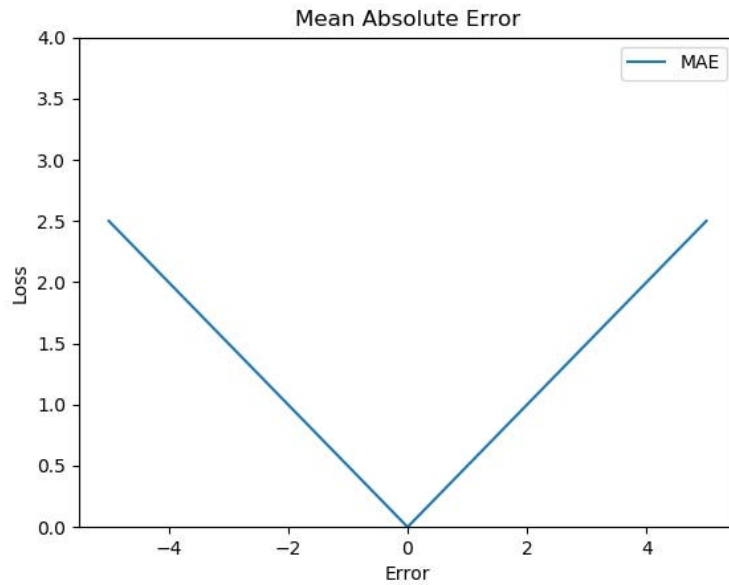


Figure 2.2: Rappresentazione grafica del MAE.

- **Mean Squared Error (MSE):** è definito come

$$MSE = \frac{1}{n} \sum_{i=1}^n \left(\hat{f}(x_i) - t_i \right)^2$$

In questa metrica, gli errori vengono elevati al quadrato, penalizzando maggiormente gli errori di grande entità.

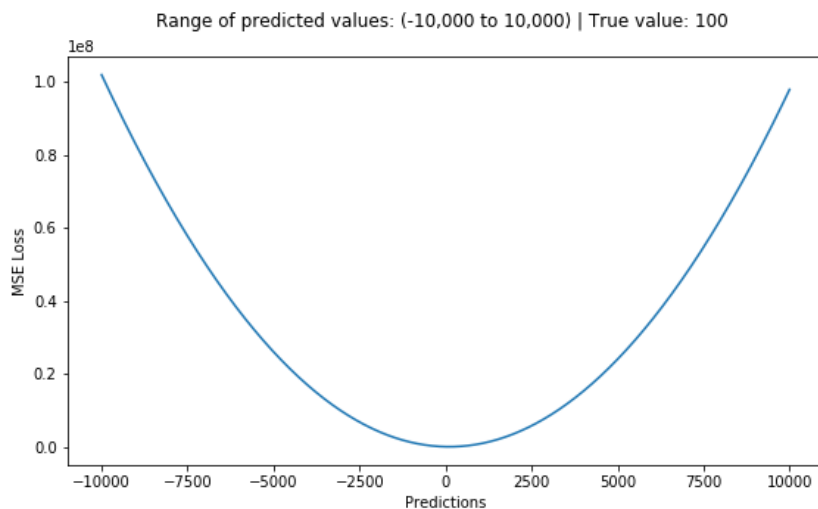


Figure 2.3: Rappresentazione grafica del MSE.

- **Root Mean Squared Error (RMSE):** è la radice quadrata del MSE:

$$RMSE = \sqrt{MSE}$$

Il RMSE mantiene la sensibilità agli errori grandi tipica del MSE, ma riporta l'errore nella stessa unità di misura del target.

Confronto tra le metriche

Le tre metriche differiscono per il modo in cui penalizzano gli errori:

- il MAE cresce linearmente con l'errore;
- il MSE cresce quadraticamente;
- il RMSE eredita la penalizzazione del MSE, ma è più interpretabile.

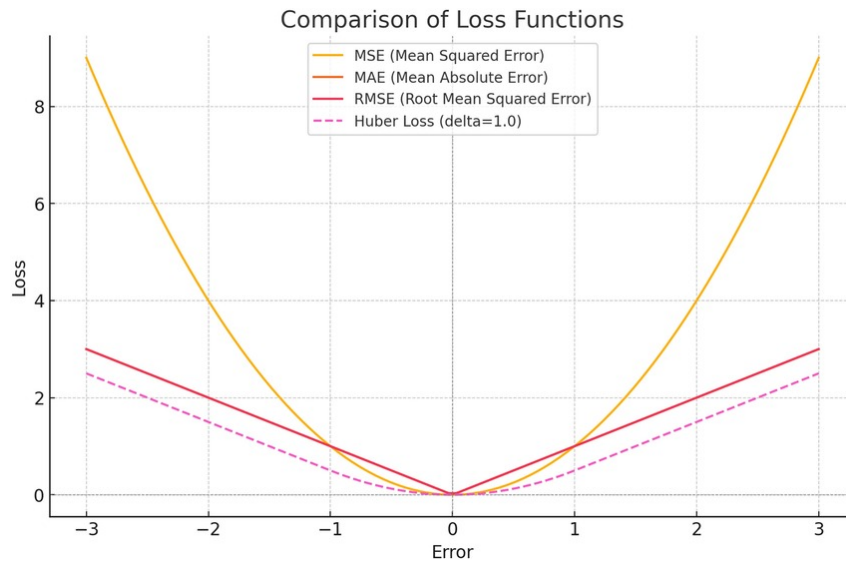


Figure 2.4: Rappresentazione del confronto tra le metriche.

2.8.1 Metriche percentuali e coefficiente di determinazione

In alcuni contesti è utile valutare l'errore in termini percentuali, ad esempio tramite il Mean Absolute Percentage Error (MAPE).

Un'ulteriore metrica importante è il coefficiente di determinazione:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - t_i)^2}{\sum_{i=1}^n (t_i - \mu)^2} \quad \text{con} \quad \mu = \frac{1}{n} \sum_{i=1}^n t_i$$

Il valore $R^2 = 0$ corrisponde a un modello che predice sempre la media dei dati.

2.8.2 Cross-validation per la regressione

La k-Fold Cross Validation può essere estesa ai problemi di regressione utilizzando metriche appropriate, come il MAE o l'RMSE. L'errore finale è ottenuto come media degli errori calcolati sui singoli fold.

Riepilogo

Valutazione delle prestazioni Stima della capacità di generalizzazione di un'ipotesi o di un algoritmo su dati non osservati.

Errore vero ed errore campionario

$$\begin{aligned} error_D(h) &= \Pr_{x \sim D}[f(x) \neq h(x)] \\ error_S(h) &= \frac{1}{|S|} \sum_{x \in S} \delta(f(x) \neq h(x)) \end{aligned}$$

Train / Test split I dati di valutazione devono essere indipendenti da quelli di training.

k-Fold Cross Validation

$$error_{L,D} = \frac{1}{k} \sum_{i=1}^k error_S(h_i)$$

Ogni esempio è usato una volta per il test e $k - 1$ volte per il training.

Overfitting

$$error_S(h) < error_S(h') \wedge error_D(h) > error_D(h')$$

Errore di training basso, errore vero alto.

Metriche per la classificazione

$$\begin{aligned} Accuracy &= \frac{TP + TN}{TP + TN + FP + FN} \\ Precision &= \frac{TP}{TP + FP} \quad Recall = \frac{TP}{TP + FN} \\ F1 &= \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \end{aligned}$$

ROC: TPR vs FPR — AUC misura la capacità discriminante.

Metriche per la regressione

$$\begin{aligned} MAE &= \frac{1}{n} \sum_{i=1}^n \left| \hat{f}(x_i) - t_i \right| \\ MSE &= \frac{1}{n} \sum_{i=1}^n \left(\hat{f}(x_i) - t_i \right)^2 \\ RMSE &= \sqrt{MSE} \\ R^2 &= 1 - \frac{\sum_{i=1}^n (\hat{f}(x_i) - t_i)^2}{\sum_{i=1}^n (t_i - \mu)^2} \end{aligned}$$

Chapter 3

Decision Tree

3.1 Definizione

Un *Decision Tree* è un modello di apprendimento supervisionato che rappresenta una funzione di decisione sotto forma di struttura ad albero. Il processo decisionale avviene mediante una sequenza di test su attributi che, partendo dalla radice, conducono a una foglia contenente la predizione finale.

Formalmente, un decision tree implementa una funzione:

$$h : X \rightarrow Y$$

dove X rappresenta lo spazio delle istanze descritte da un insieme di attributi e Y lo spazio delle etichette di output. Ogni cammino dalla radice a una foglia può essere interpretato come una regola di decisione del tipo:

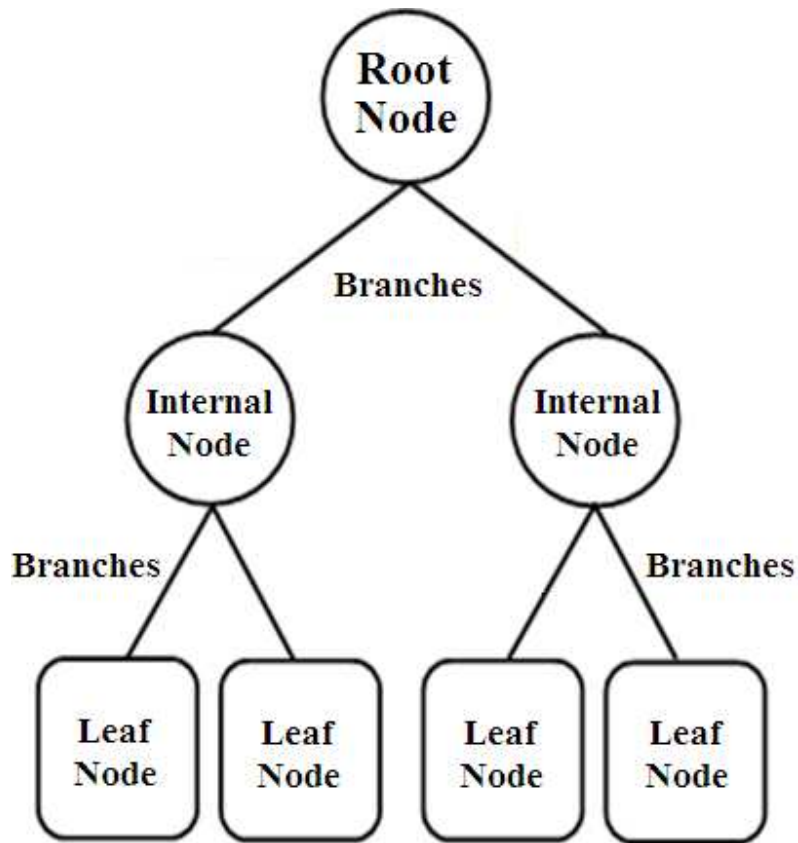
$$\text{IF } (\text{condizione}_1 \wedge \text{condizione}_2 \wedge \dots) \Rightarrow y$$

3.1.1 Struttura del Decision Tree

Un decision tree è composto dai seguenti elementi fondamentali:

- **Nodo radice**, che rappresenta il primo test su un attributo;
- **Nodi interni**, ciascuno dei quali corrisponde a un test su un attributo;
- **Archi**, che rappresentano i possibili esiti del test;
- **Foglie**, che contengono la decisione finale del modello.

Nel caso di problemi di classificazione, ogni foglia è associata a una classe, mentre nei problemi di regressione la foglia contiene un valore reale.



(a)

Figure 3.1: Struttura generale di un decision tree: nodo radice, nodi interni e foglie.

3.1.2 Caratteristiche principali

I decision tree presentano diverse proprietà che li rendono modelli particolarmente utilizzati:

- **Interpretabilità:** il modello è facilmente interpretabile, poiché il percorso seguito per effettuare una predizione è esplicito;
- **Assenza di assunzioni forti:** non richiedono ipotesi sulla distribuzione dei dati o relazioni lineari tra gli attributi;
- **Flessibilità:** possono gestire sia attributi discreti che continui;
- **Apprendimento non parametrico:** la complessità del modello cresce con i dati;
- **Rischio di overfitting:** alberi troppo profondi possono adattarsi eccessivamente ai dati di training, riducendo la capacità di generalizzazione.

Esempio

Si consideri un problema di classificazione in cui l'obiettivo è decidere se giocare a tennis in base alle condizioni meteorologiche. Gli attributi considerati sono:

- $Outlook \in \{Sunny, Overcast, Rain\}$;
- $Humidity \in \{High, Normal\}$;
- $Wind \in \{Weak, Strong\}$.

La classe di output è:

$$PlayTennis \in \{Yes, No\}$$

Un possibile decision tree è il seguente:

- se $Outlook = Overcast$, allora $PlayTennis = Yes$;
- se $Outlook = Sunny$:
 - se $Humidity = High$, allora $PlayTennis = No$;
 - se $Humidity = Normal$, allora $PlayTennis = Yes$;
- se $Outlook = Rain$:
 - se $Wind = Strong$, allora $PlayTennis = No$;
 - se $Wind = Weak$, allora $PlayTennis = Yes$.

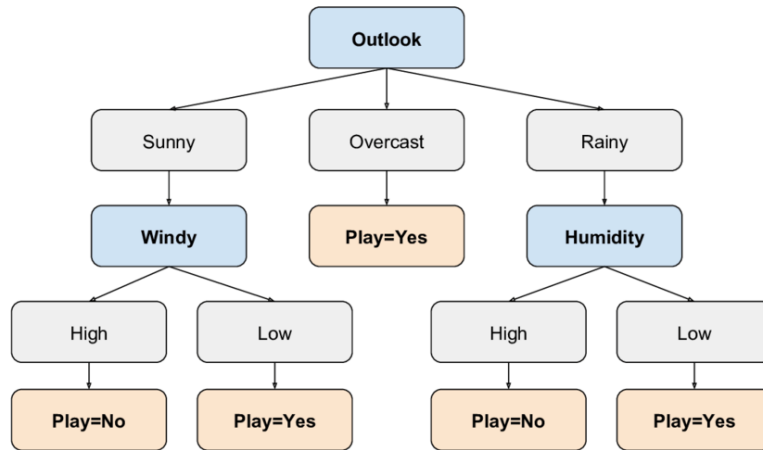


Figure 3.2: Esempio di decision tree per il problema Play Tennis.

3.2 Regole di decisione

Nel contesto dei decision tree, una **regola di decisione** è una proposizione logica che associa una combinazione di condizioni sugli attributi a una decisione finale.

Formalmente, una regola ha la forma:

$$\text{IF } (condizione_1 \wedge condizione_2 \wedge \dots \wedge condizione_k) \Rightarrow y$$

dove ciascuna condizione rappresenta un test su un attributo e y è il valore di output associato.

3.2.1 Decision Tree come insieme di regole

Un decision tree può essere interpretato come un insieme di regole di decisione. Ogni cammino che parte dal nodo radice e termina in una foglia definisce una regola distinta.

Pertanto, un decision tree rappresenta una funzione di decisione equivalente a una congiunzione di regole del tipo:

$$h(x) = \bigvee_{i=1}^m r_i(x)$$

dove ciascuna regola r_i è associata a una foglia dell'albero.

3.2.2 Conversione di un decision tree in regole

La conversione di un decision tree in un insieme di regole avviene secondo i seguenti passaggi:

1. si seleziona una foglia dell'albero;
2. si considera il cammino dalla radice a tale foglia;
3. per ogni nodo attraversato, si aggiunge alla regola la condizione associata all'arco seguito;
4. la classe associata alla foglia costituisce la conclusione della regola.

Ripetendo la procedura per tutte le foglie si ottiene un insieme di regole logicamente equivalenti al decision tree originale.

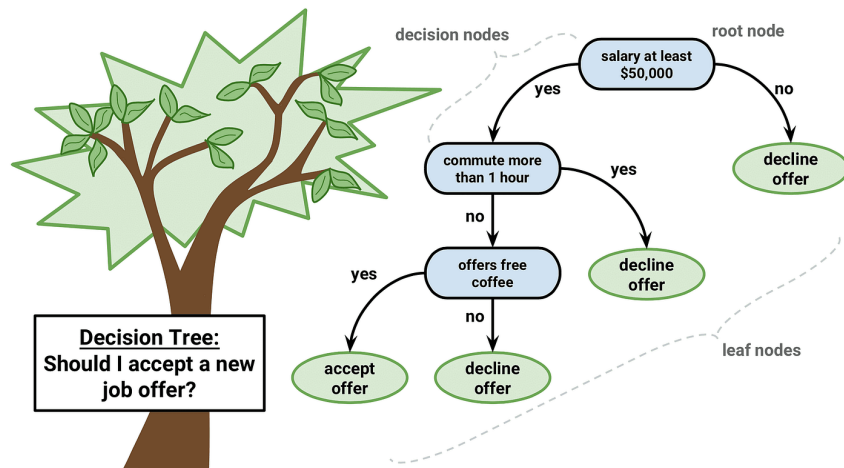


Figure 3.3: Conversione di un decision tree in un insieme di regole: ogni percorso radice–foglia corrisponde a una regola.

Esempio

Si consideri il decision tree del problema *Play Tennis*. Uno dei possibili cammini radice–foglia è il seguente:

$$\text{Outlook} = \text{Sunny} \wedge \text{Humidity} = \text{High}$$

La regola di decisione corrispondente è:

$$\text{IF } (\text{Outlook} = \text{Sunny} \wedge \text{Humidity} = \text{High}) \Rightarrow \text{PlayTennis} = \text{No}$$

Ripetendo il procedimento per ciascuna foglia dell'albero si ottiene l'insieme completo delle regole che descrivono il modello.

Osservazione

Un decision tree e l'insieme di regole da esso derivato sono logicamente equivalenti. Tuttavia, la rappresentazione a regole consente una maggiore flessibilità nella semplificazione del modello, ad esempio tramite l'eliminazione di condizioni non necessarie.

3.3 Algoritmo ID3

L'algoritmo *ID3* (Iterative Dichotomiser 3) è uno dei primi algoritmi proposti per l'apprendimento di decision tree a partire da dati etichettati. ID3 costruisce l'albero in modo incrementale, adottando una strategia **greedy top-down**, in cui ad ogni passo viene selezionato l'attributo che massimizza un criterio di bontà dello split.

L'idea alla base dell'algoritmo ID3 è quella di costruire l'albero selezionando, a ogni nodo, l'attributo che consente di separare al meglio gli esempi di training in base alle classi.

La costruzione dell'albero avviene:

- partendo da tutti gli esempi nel nodo radice;
- scegliendo l'attributo più informativo;
- suddividendo il dataset in sottoinsiemi;
- ripetendo ricorsivamente il procedimento su ciascun sottoinsieme.

3.3.1 Costruzione del decision tree

Sia S un insieme di esempi di training e sia A l'insieme degli attributi disponibili. L'algoritmo ID3 costruisce il decision tree secondo i seguenti passi:

1. Se tutti gli esempi in S appartengono alla stessa classe, crea una foglia etichettata con tale classe.
2. Se l'insieme degli attributi A è vuoto, crea una foglia etichettata con la classe più frequente in S .
3. Altrimenti, seleziona l'attributo $a \in A$ che massimizza il criterio di *Information Gain*.
4. Crea un nodo interno etichettato con l'attributo selezionato.
5. Per ciascun valore possibile v dell'attributo a :
 - crea un ramo corrispondente al test $a = v$;
 - considera il sottoinsieme $S_v \subseteq S$ degli esempi per cui $a = v$;
 - applica ricorsivamente l'algoritmo ID3 a S_v e all'insieme $A \setminus \{a\}$.

Condizioni di arresto

La crescita dell'albero termina quando si verifica almeno una delle seguenti condizioni:

- tutti gli esempi nel nodo appartengono alla stessa classe;
- non sono disponibili ulteriori attributi per effettuare lo split;
- il sottoinsieme di esempi associato al nodo è vuoto.

Pseudocodice

```
ID3(S, A):  
    if tutti gli esempi in S hanno la stessa classe:  
        return foglia con quella classe  
    if A è vuoto:  
        return foglia con la classe più frequente in S  
    seleziona a in A che massimizza Information Gain  
    crea un nodo con test sull'attributo a  
    for ogni valore v di a:  
        S_v = sottoinsieme di S con a = v  
        figlio = ID3(S_v, A \ {a})  
        collega figlio al nodo tramite ramo a = v  
    return nodo
```

3.3.2 Proprietà dell'algoritmo ID3

L'algoritmo ID3 presenta le seguenti caratteristiche:

- adotta una strategia greedy e non garantisce l'albero ottimo globale;
- utilizza solo attributi discreti;
- non prevede tecniche di pruning;
- è sensibile al rumore nei dati;
- tende a costruire alberi completi che possono soffrire di overfitting.

3.3.3 ID3 e regole di decisione

L'albero prodotto dall'algoritmo ID3 può essere convertito in un insieme di regole di decisione. Ogni foglia dell'albero corrisponde a una regola ottenuta congiungendo le condizioni lungo il cammino radice-foglia.

3.4 Entropia e Information Gain

Nell'algoritmo ID3 è necessario stabilire quale attributo utilizzare per suddividere gli esempi in ciascun nodo dell'albero. A tal fine viene introdotta una misura quantitativa della **purezza** di un insieme di esempi, basata su concetti di teoria dell'informazione.

3.4.1 Entropia

Sia S un insieme di esempi di training appartenenti a c classi distinte. Si indichi con p_i la proporzione di esempi di S appartenenti alla classe i .

L'entropia di S è definita come:

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2 p_i$$

L'entropia misura il grado di incertezza o impurità dell'insieme di esempi ed assume i seguenti valori caratteristici:

- $Entropy(S) = 0$ se tutti gli esempi appartengono alla stessa classe;
- $Entropy(S)$ è massima quando le classi sono equamente distribuite;
- valori intermedi indicano un grado parziale di impurità.

Pertanto, un insieme di esempi è tanto più informativo quanto minore è la sua entropia.

Esempio

Si consideri un insieme S composto da 10 esempi, di cui 5 positivi e 5 negativi. In questo caso:

$$p_+ = 0.5 \quad p_- = 0.5$$

L'entropia risulta:

$$Entropy(S) = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$$

Se invece tutti gli esempi appartenessero alla stessa classe, l'entropia sarebbe nulla.

3.4.2 Information Gain

L'**Information Gain** misura la riduzione di entropia ottenuta suddividendo un insieme di esempi in base a un attributo.

Dato un insieme di esempi S e un attributo A con valori possibili $\{v_1, v_2, \dots, v_k\}$, l'Information Gain di A rispetto a S è definito come:

$$Gain(S, A) = Entropy(S) - \sum_{j=1}^k \frac{|S_{v_j}|}{|S|} Entropy(S_{v_j})$$

dove $S_{v_j} \subseteq S$ è il sottoinsieme degli esempi per cui l'attributo A assume valore v_j .

L'Information Gain rappresenta la quantità di informazione ottenuta conoscendo il valore di un attributo. Un attributo è tanto più informativo quanto maggiore è la riduzione di entropia che produce.

Pertanto, l'attributo che massimizza l'Information Gain è quello che separa meglio gli esempi in sottoinsiemi più puri.

Utilizzo in ID3

Nell'algoritmo ID3, la selezione dell'attributo da utilizzare in un nodo avviene scegliendo quello che massimizza l'Information Gain.

Formalmente, dato l'insieme degli attributi disponibili A , viene selezionato:

$$a^* = \arg \max_{a \in A} \text{Gain}(S, a)$$

L'attributo selezionato viene utilizzato per suddividere il dataset e il procedimento viene applicato ricorsivamente ai sottoinsiemi generati.

3.4.3 Schema operativo

La costruzione dell'albero tramite ID3 può essere riassunta come segue:

1. calcolare l'entropia dell'insieme di esempi corrente;
2. calcolare l'Information Gain per ciascun attributo disponibile;
3. selezionare l'attributo con massimo Information Gain;
4. suddividere il dataset in base ai valori dell'attributo selezionato;
5. ripetere ricorsivamente il procedimento sui sottoinsiemi ottenuti.

3.5 Overfitting nei Decision Tree

I decision tree sono modelli altamente flessibili e possono adattarsi in modo molto accurato ai dati di training. Tuttavia, questa flessibilità comporta un elevato rischio di **overfitting**.

Un decision tree presenta overfitting quando continua a crescere fino a classificare correttamente tutti (o quasi) gli esempi di training, includendo anche il rumore presente nei dati. In tali casi, l'albero risulta molto accurato sui dati di training ma generalizza male su dati non osservati.

Definizione formale

Dato un insieme di esempi di training S , un decision tree h presenta overfitting se esiste un altro albero h' tale che:

$$\text{error}_S(h) < \text{error}_S(h') \wedge \text{error}_D(h) > \text{error}_D(h')$$

In altre parole, h è migliore sui dati di training, ma peggiore sui dati futuri rispetto a h' .

Cause principali

Le principali cause dell'overfitting nei decision tree sono:

- crescita eccessiva dell'albero;
- presenza di rumore nei dati di training;

- utilizzo di attributi poco informativi;
- assenza di vincoli sulla profondità dell'albero.

Poiché ID3 costruisce l'albero fino a quando è possibile separare ulteriormente gli esempi, esso tende a generare alberi completi e complessi.

3.6 Pruning

Il **pruning** è l'insieme di tecniche volte a ridurre la complessità del decision tree al fine di migliorare la capacità di generalizzazione del modello.

L'obiettivo del pruning non è minimizzare l'errore di training, bensì ridurre l'errore vero eliminando rami che modellano il rumore dei dati.

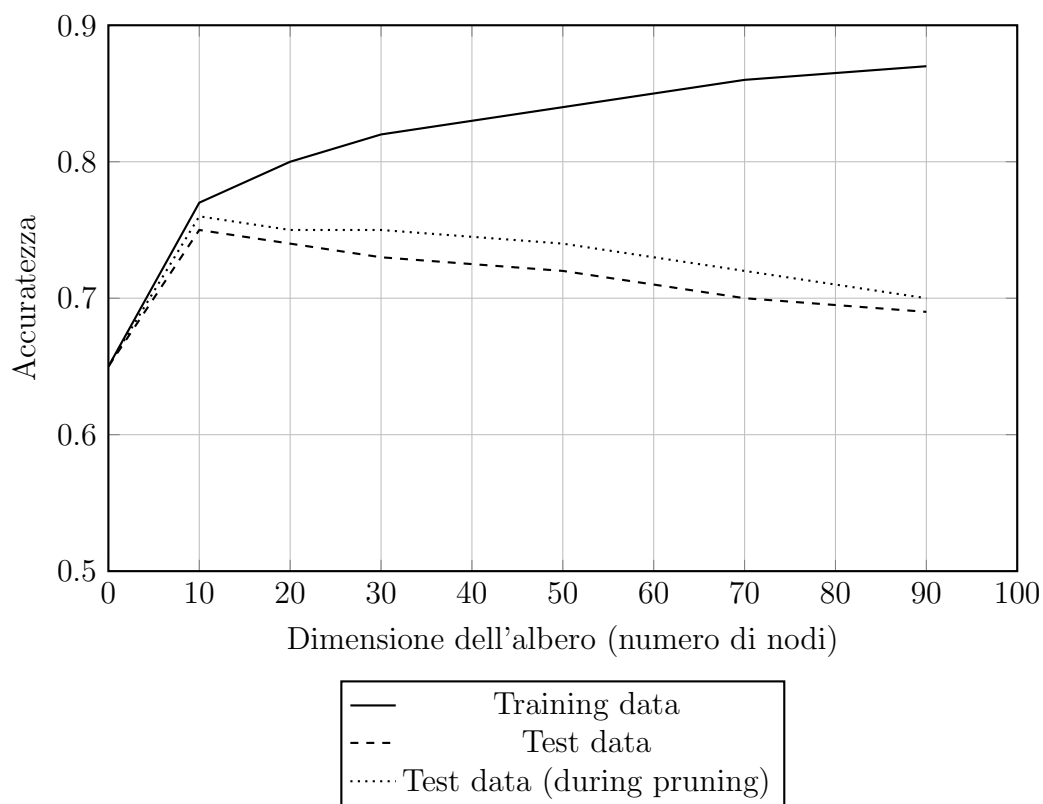


Figure 3.4: Effetto del Reduced-Error Pruning: l'accuratezza sui dati di training cresce con la complessità dell'albero, mentre l'accuratezza sui dati di test mostra un massimo per una complessità intermedia.

3.6.1 Pre-pruning

Il **pre-pruning** consiste nell'interrompere la crescita dell'albero prima che esso raggiunga una struttura completa. La decisione di arrestare la suddivisione viene presa durante la fase di costruzione dell'albero.

Criteri comuni di pre-pruning includono:

- profondità massima dell'albero;
- numero minimo di esempi in un nodo;

- Information Gain inferiore a una soglia;
- assenza di miglioramento significativo delle prestazioni.

Il pre-pruning riduce il rischio di overfitting e il costo computazionale, ma può introdurre underfitting se applicato in modo eccessivo.

3.6.2 Post-pruning

Il **post-pruning** consiste nel costruire inizialmente un decision tree completo e successivamente semplificarlo eliminando rami poco significativi.

Il procedimento tipico di post-pruning prevede:

- la valutazione delle prestazioni di un sottoalbero su un validation set;
- la sostituzione del sottoalbero con una foglia;
- il confronto tra le prestazioni prima e dopo la potatura;
- il mantenimento della potatura solo se migliora la generalizzazione.

Il post-pruning risulta generalmente più efficace del pre-pruning, poiché consente di valutare l'impatto reale delle potature sulla capacità di generalizzazione.

Confronto Pre-Pruning vs Post-Pruning

Metodo	Quando applicato	Rischio principale
Pre-pruning	Durante la crescita	Underfitting
Post-pruning	Dopo la crescita	Costo computazionale

3.7 Random Forest

Il *Random Forest* è un metodo di apprendimento basato su un insieme di decision tree, appartenente alla famiglia dei metodi di **ensemble**. L'idea di base è quella di combinare le predizioni di più alberi di decisione al fine di ottenere un modello più robusto e con migliori capacità di generalizzazione.

Idea fondamentale

Un singolo decision tree presenta elevata varianza ed è particolarmente sensibile ai dati di training. Il Random Forest riduce questo problema costruendo molti alberi diversi e aggregandone le predizioni.

Ogni albero viene addestrato:

- su un sottoinsieme casuale degli esempi di training (bootstrap);
- utilizzando, a ogni nodo, un sottoinsieme casuale degli attributi disponibili.

3.7.1 Aggregazione delle predizioni

Nel caso di classificazione, la predizione finale del Random Forest è ottenuta tramite voto di maggioranza tra le predizioni dei singoli alberi. Nel caso di regressione, la predizione finale è data dalla media delle predizioni.

Formalmente, dati $h_1, h_2, \dots, h_T$ decision tree:

$$\hat{y} = \text{majority_vote}\{h_1(x), \dots, h_T(x)\}$$

3.7.2 Riduzione dell'overfitting

Il Random Forest riduce l'overfitting combinando modelli ad alta varianza ma a basso bias. L'introduzione di casualità nella costruzione degli alberi rende le predizioni meno correlate tra loro, migliorando la stabilità del modello complessivo.

A differenza dei singoli decision tree, il Random Forest tende a generalizzare meglio senza richiedere un pruning esplicito.

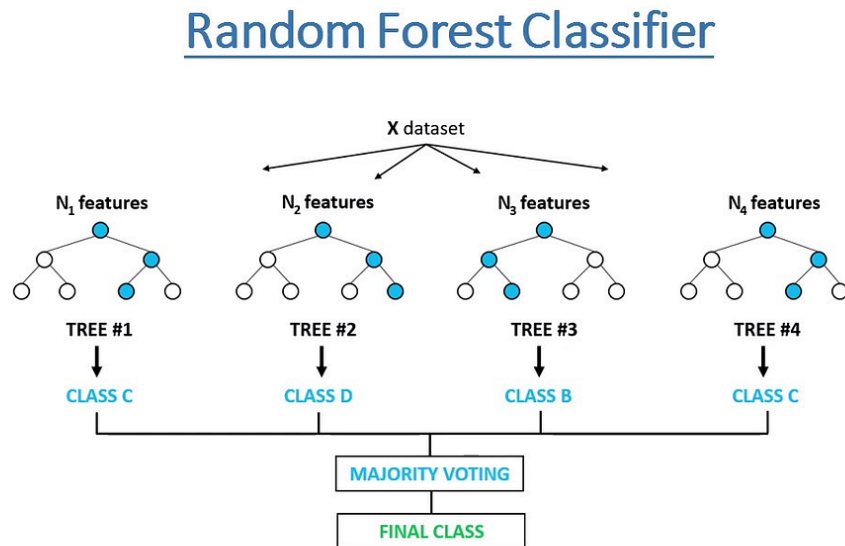


Figure 3.5: Schema concettuale di un Random Forest: più decision tree addestrati su sottoinsiemi diversi dei dati e degli attributi contribuiscono alla predizione finale.

Riepilogo

Decision Tree Modello supervisionato che rappresenta una funzione di decisione sotto forma di struttura ad albero. Ogni cammino radice–foglia corrisponde a una regola di decisione del tipo IF–THEN.

Regole di decisione Un decision tree è logicamente equivalente a un insieme di regole. Ogni foglia dell'albero definisce una regola ottenuta congiungendo le condizioni lungo il percorso radice–foglia.

Algoritmo ID3 Costruisce l'albero in modo greedy e top-down. A ogni nodo seleziona l'attributo che massimizza l'Information Gain e suddivide ricorsivamente il dataset. Non garantisce l'albero ottimo globale e non prevede tecniche di pruning.

Entropia

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2 p_i$$

Misura l'impurità di un insieme di esempi. È nulla se tutti gli esempi appartengono alla stessa classe ed è massima quando le classi sono equamente distribuite.

Information Gain

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Misura la riduzione di entropia ottenuta suddividendo gli esempi in base a un attributo. ID3 seleziona l'attributo con massimo Information Gain.

Overfitting Un decision tree può adattarsi eccessivamente ai dati di training, ottenendo errore campionario basso ma scarsa capacità di generalizzazione:

$$error_S(h) < error_S(h') \wedge error_D(h) > error_D(h')$$

Pruning Tecniche volte a ridurre la complessità dell'albero per migliorare la generalizzazione.

- *Pre-pruning*: arresto anticipato della crescita (rischio di underfitting);
- *Post-pruning*: potatura di un albero completo sulla base delle prestazioni su validation set.

Random Forest Metodo ensemble che combina più decision tree addestrati su sottoinsiemi casuali di dati e attributi. Riduce la varianza e l'overfitting dei singoli alberi, a costo di una minore interpretabilità.

RIEPILOGO — Probabilità e Bayes Networks

Incertezza In ambienti reali l'informazione è incompleta, rumorosa e parzialmente osservabile. Un approccio puramente logico risulta inadeguato per prendere decisioni efficaci.

Probabilità La probabilità fornisce una rappresentazione quantitativa dell'incertezza. Uno spazio di probabilità è definito da:

$$\Omega \text{ (spazio campionario)}, \quad P : \Omega \rightarrow [0, 1], \quad \sum_{\omega \in \Omega} P(\omega) = 1$$

Eventi e variabili casuali Un evento è un sottoinsieme di Ω . Una variabile casuale è una funzione:

$$X : \Omega \rightarrow D(X)$$

che induce una distribuzione di probabilità $P(X = x_i)$.

Distribuzioni di probabilità

- *Distribuzione prior*: credenza prima di osservare evidenza;
- *Distribuzione congiunta*: probabilità di tutte le combinazioni di valori delle variabili;
- *Distribuzione condizionata*: probabilità di una variabile dato il valore di un'altra.

Probabilità condizionata e regola del prodotto

$$P(a \mid b) = \frac{P(a \wedge b)}{P(b)}, \quad P(a, b) = P(a \mid b)P(b)$$

Probabilità totali

$$P(X) = \sum_{y \in D(Y)} P(X \mid Y = y)P(Y = y)$$

Chain Rule

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid X_1, \dots, X_{i-1})$$

Indipendenza Due eventi sono indipendenti se:

$$P(A, B) = P(A)P(B)$$

L'indipendenza assoluta è rara nei sistemi complessi.

Indipendenza condizionata

$$X \perp Y \mid Z \iff P(X \mid Y, Z) = P(X \mid Z)$$

Consente una rappresentazione compatta delle distribuzioni congiunte.

Regola di Bayes

$$P(Y | X) = \frac{P(X | Y)P(Y)}{P(X)} = \alpha P(X | Y)P(Y)$$

Fondamentale per il ragionamento diagnostico (cause a partire dagli effetti).

Massima verosimiglianza

$$\arg \max_{x_i \in D(X)} P(X = x_i | Y) = \arg \max_{x_i} P(Y | X = x_i)P(X = x_i)$$

Bayesian Networks Modelli grafici che rappresentano variabili casuali e relazioni di dipendenza tramite:

- nodi (variabili);
- archi diretti (dipendenze causali);
- CPT: $P(X_i | Parents(X_i))$.

Le reti bayesiane permettono di rappresentare distribuzioni congiunte in modo compatto, riducendo la complessità da esponenziale a lineare (a parità di genitori).

Chapter 4

Bayesian Learning

4.1 Motivazione

Nel Machine Learning supervisionato, l'obiettivo principale è apprendere una funzione che consenta di effettuare predizioni corrette su nuove istanze non osservate. Nei capitoli precedenti, l'apprendimento è stato affrontato prevalentemente come un problema di ricerca nello spazio delle ipotesi, in cui si seleziona una singola ipotesi che ottimizza una misura di performance sui dati di training.

Tuttavia, in molti contesti reali l'informazione disponibile è incompleta, rumorosa e soggetta a incertezza. In questi casi, risulta limitante adottare un approccio deterministico che restituisca un'unica ipotesi o una singola predizione senza fornire indicazioni sul grado di confidenza associato.

Il **Bayesian Learning** affronta il problema dell'apprendimento introducendo una rappresentazione esplicita dell'incertezza tramite il calcolo delle probabilità. Invece di selezionare una singola ipotesi, l'approccio bayesiano considera una distribuzione di probabilità sulle ipotesi e utilizza i dati osservati per aggiornare tale distribuzione.

In questo quadro, l'apprendimento non consiste esclusivamente nella scelta di un modello, ma nella stima di quanto ogni possibile ipotesi sia plausibile alla luce dei dati disponibili.

4.2 Classificazione come stima probabilistica

Dal punto di vista bayesiano, la classificazione può essere interpretata come un problema di stima probabilistica. Dato un dataset di training D e una nuova istanza x' , l'obiettivo non è semplicemente assegnare un'etichetta, ma stimare la distribuzione di probabilità sulle possibili classi.

Sia $f : X \rightarrow V$ la funzione target, dove X è lo spazio delle istanze e V l'insieme dei possibili valori di output. La miglior predizione per una nuova istanza x' è definita come:

$$\hat{f}(x') = v^* \quad \text{con} \quad v^* = \arg \max_{v \in V} P(v \mid x', D)$$

In forma più generale, l'obiettivo del Bayesian Learning è stimare l'intera distribuzione:

$$P(V \mid x', D)$$

Questa formulazione consente di quantificare l'incertezza associata alle predizioni e costituisce la base per l'introduzione dei classificatori bayesiani.

4.3 ML e MAP

Dato un dataset di training D e una nuova istanza x' , l'obiettivo del Bayesian Learning è stimare la probabilità a posteriori delle classi:

$$P(v \mid x', D)$$

Utilizzando il teorema di Bayes, tale probabilità può essere riscritta come:

$$P(v \mid x', D) = \frac{P(x' \mid v, D) P(v \mid D)}{P(x' \mid D)}$$

Poiché il termine $P(x' \mid D)$ non dipende dalla classe v , esso può essere ignorato nel confronto tra classi.

4.3.1 Maximum Likelihood Classification

Nel criterio di **Maximum Likelihood (ML)**, si assume che tutte le classi siano ugualmente probabili a priori, ovvero:

$$P(v \mid D) = \text{costante}$$

In questo caso, la classificazione si riduce alla massimizzazione della verosimiglianza:

$$v_{ML} = \arg \max_{v \in V} P(x' \mid v, D)$$

Il classificatore ML assegna quindi a x' la classe che rende l'osservazione più probabile, indipendentemente dalla frequenza delle classi nel dataset.

Questo criterio quindi seleziona la classe che meglio spiega l'osservazione corrente. Tuttavia, esso non tiene conto di alcuna informazione a priori sulla distribuzione delle classi. Di conseguenza, il ML può risultare inadeguato in presenza di classi sbilanciate o quando il numero di esempi di training è limitato.

4.3.2 Maximum A Posteriori Classification

Il criterio di **Maximum A Posteriori (MAP)** estende il ML introducendo l'informazione a priori sulle classi. La classificazione avviene selezionando la classe che massimizza la probabilità a posteriori:

$$v_{MAP} = \arg \max_{v \in V} P(v \mid x', D)$$

Applicando il teorema di Bayes e trascurando il termine di normalizzazione, si ottiene:

$$v_{MAP} = \arg \max_{v \in V} P(x' \mid v, D) P(v \mid D)$$

Il criterio MAP combina due informazioni:

- la verosimiglianza $P(x' \mid v, D)$, che misura quanto bene la classe spiega l'osservazione;
- la probabilità a priori $P(v \mid D)$, che riflette la frequenza o plausibilità della classe.

In questo modo, il MAP penalizza classi rare e favorisce classi più probabili a priori, risultando più robusto rispetto al ML in presenza di dati sbilanciati.

Confronto tra ML e MAP

Criterio	Funzione obiettivo	Informazione a priori
ML	$\arg \max_v P(x' v, D)$	No
MAP	$\arg \max_v P(x' v, D) P(v D)$	Sì

Osservazione

Il criterio ML può essere visto come un caso particolare del MAP in cui tutte le classi hanno probabilità a priori uguali. L'introduzione di informazioni a priori consente al MAP di migliorare la qualità delle predizioni in contesti realistici.

4.4 Bayes Optimal Classifier

Nel contesto del Bayesian Learning, i criteri ML e MAP selezionano una singola classe sulla base di una stima puntuale delle probabilità. Tuttavia, tali approcci assumono implicitamente di conoscere con certezza il modello probabilistico sottostante.

Il **Bayes Optimal Classifier** rappresenta il classificatore teoricamente ottimale, in quanto tiene conto di *tutte* le ipotesi possibili pesate dalla loro probabilità a posteriori. In un approccio completamente bayesiano, infatti, non si seleziona una singola ipotesi h , ma si considera l'intera distribuzione di probabilità sulle ipotesi dato il dataset di training. La classificazione ottimale si ottiene combinando le predizioni di tutte le ipotesi, ciascuna pesata dalla propria probabilità a posteriori.

Definizione formale

Sia H lo spazio delle ipotesi e sia D il dataset di training. La probabilità che una nuova istanza x' appartenga alla classe v è data da:

$$P(v | x', D) = \sum_{h \in H} P(v | x', h) P(h | D)$$

Il **Bayes Optimal Classifier** assegna a x' la classe che massimizza tale probabilità:

$$v^* = \arg \max_{v \in V} \sum_{h \in H} P(v | x', h) P(h | D)$$

Questo metodo minimizza l'errore di classificazione atteso rispetto alla distribuzione a posteriori sulle ipotesi. Esso rappresenta quindi il limite superiore delle prestazioni ottenibili da qualsiasi classificatore basato sulle stesse assunzioni probabilistiche.

In pratica, tuttavia, il calcolo esplicito del Bayes Optimal Classifier è spesso intrattabile, poiché richiede di considerare tutte le ipotesi nello spazio H .

Esempio

Si consideri un problema di classificazione binaria con due ipotesi h_1 e h_2 . Supponiamo che, dato il dataset D , le probabilità a posteriori delle ipotesi siano:

$$P(h_1 | D) = 0.6 \quad P(h_2 | D) = 0.4$$

Per una nuova istanza x' , le ipotesi forniscono le seguenti predizioni:

$$P(v = 1 \mid x', h_1) = 0.9 \quad P(v = 1 \mid x', h_2) = 0.2$$

Il Bayes Optimal Classifier calcola:

$$P(v = 1 \mid x', D) = 0.9 \cdot 0.6 + 0.2 \cdot 0.4 = 0.62$$

$$P(v = 0 \mid x', D) = 1 - 0.62 = 0.38$$

Pertanto, la classe assegnata è $v = 1$, ottenuta combinando le predizioni di entrambe le ipotesi.

Osservazione

Il Bayes Optimal Classifier rappresenta il classificatore teoricamente migliore in termini di errore atteso. Nella pratica, modelli come MAP e Naive Bayes possono essere interpretati come approssimazioni computazionalmente efficienti di questo approccio ideale.

4.5 Naive Bayes Classifier

4.5.1 Definizione

Il *Naive Bayes Classifier* è un classificatore probabilistico basato sul teorema di Bayes e rappresenta un'approssimazione pratica del Bayes Optimal Classifier.

Il Bayes Optimal Classifier infatti richiede di considerare tutte le ipotesi possibili nello spazio H , risultando spesso intrattabile dal punto di vista computazionale. Il Naive Bayes supera questo problema assumendo che gli attributi siano condizionatamente indipendenti dato il valore della classe.

Formalmente, se un'istanza è descritta da $x = (x_1, x_2, \dots, x_n)$, si assume che:

$$P(x_1, x_2, \dots, x_n \mid v) = \prod_{i=1}^n P(x_i \mid v)$$

Questa assunzione implica che, fissata la classe, il valore di ciascun attributo non fornisce informazioni sugli altri attributi. In altre parole, tutta la dipendenza tra gli attributi è mediata dalla variabile di classe.

Nella maggior parte dei problemi reali però, gli attributi non sono realmente indipendenti. Ad esempio, nel testo di un documento o nelle caratteristiche di un'immagine, la presenza di una certa informazione rende più probabile la presenza di altre. Pertanto, l'assunzione di indipendenza condizionata è raramente soddisfatta in modo rigoroso.

Nonostante l'assunzione di indipendenza sia spesso violata, il Naive Bayes mostra buone prestazioni in molti contesti pratici. Ciò è dovuto a diversi fattori:

- l'obiettivo della classificazione è confrontare probabilità relative tra classi, non stimare probabilità assolute accurate;
- errori sistematici nelle stime delle probabilità tendono a compensarsi nel confronto tra classi;

- il modello ha bassa varianza, risultando particolarmente robusto in presenza di pochi dati di training;
- la semplicità del modello riduce il rischio di overfitting.

Di conseguenza, anche con stime probabilistiche imprecise, il Naive Bayes è spesso in grado di individuare correttamente la classe più probabile.

4.5.2 Formulazione formale

Sia $x = (x_1, x_2, \dots, x_n)$ una istanza descritta da n attributi e sia $v \in V$ una possibile classe. L'obiettivo è stimare la probabilità a posteriori:

$$P(v \mid x)$$

Applicando il teorema di Bayes, si ottiene:

$$P(v \mid x) = \frac{P(x \mid v) P(v)}{P(x)}$$

Poiché il termine $P(x)$ non dipende dalla classe, esso può essere ignorato nel confronto tra classi.

Utilizzando l'assunzione di indipendenza, la regola di classificazione del Naive Bayes diventa:

$$v_{NB} = \arg \max_{v \in V} P(v) \prod_{i=1}^n P(x_i \mid v)$$

Il classificatore assegna all'istanza x la classe che massimizza il prodotto tra la probabilità a priori della classe e le probabilità condizionate degli attributi.

Algoritmo Naive Bayes

```

Target function f : X -> V
X = A1 x A2 x ... x An
V = {v1, v2, ..., vk}
Training set D
New instance x = <a1, a2, ..., an>

Naive_Bayes_Learn(A, V, D)
  for each class vj in V
    estimate P(vj | D)
    for each attribute Ak
      for each possible value ai of Ak
        estimate P(ai | vj, D)

Classify_New_Instance(x)
  return vNB = argmax_vj P(vj | D) *
    product over ai in x of P(ai | vj, D)

```

L'algoritmo Naive Bayes si articola in due fasi distinte: una fase di apprendimento e una fase di classificazione.

Durante la fase di apprendimento (`Naive_Bayes_Learn`), il modello stima:

- la probabilità a priori di ciascuna classe $P(v_j \mid D)$, calcolata come frequenza relativa della classe nel dataset;
- le probabilità condizionate $P(a_i \mid v_j, D)$, che descrivono la distribuzione dei valori degli attributi all'interno di ciascuna classe.

Nella fase di classificazione (**Classify_New_Instance**), per una nuova istanza x , il classificatore calcola per ogni classe il prodotto tra la probabilità a priori e le probabilità condizionate degli attributi, secondo l'assunzione di indipendenza. La classe assegnata è quella che massimizza tale quantità.

Osservazione

Lo pseudocodice mostra esplicitamente come il Naive Bayes implementi la regola di decisione:

$$v_{NB} = \arg \max_{v \in V} P(v) \prod_{i=1}^n P(x_i \mid v)$$

L'algoritmo non apprende una struttura complessa, ma si limita a stimare un numero limitato di parametri probabilistici, rendendolo semplice ed efficiente.

4.5.3 Stima delle probabilità

Le probabilità utilizzate dal Naive Bayes vengono stimate a partire dal dataset di training:

- $P(v)$: frequenza relativa della classe v ;
- $P(x_i \mid v)$: frequenza dell'attributo x_i all'interno della classe v .

Nel caso di attributi continui, si assume spesso una distribuzione parametrica (ad esempio gaussiana) per modellare $P(x_i \mid v)$.

Esempio

Si consideri un problema di classificazione con due classi $V = \{\text{Spam}, \text{Ham}\}$ e due attributi binari:

$$x = (\text{contains_offer}, \text{contains_link})$$

Supponiamo di aver stimato:

$$P(\text{Spam}) = 0.4 \quad P(\text{Ham}) = 0.6$$

$$P(\text{contains_offer} = 1 \mid \text{Spam}) = 0.8$$

$$P(\text{contains_link} = 1 \mid \text{Spam}) = 0.7$$

$$P(\text{contains_offer} = 1 \mid \text{Ham}) = 0.1$$

$$P(\text{contains_link} = 1 \mid \text{Ham}) = 0.2$$

Per un nuovo messaggio con $x = (1, 1)$, il Naive Bayes calcola:

$$P(\text{Spam} \mid x) \propto 0.4 \cdot 0.8 \cdot 0.7 = 0.224$$

$$P(\text{Ham} \mid x) \propto 0.6 \cdot 0.1 \cdot 0.2 = 0.012$$

Pertanto, il messaggio viene classificato come *Spam*.

Limiti

I limiti del Naive Bayes possono essere riassunti come un compromesso tra semplicità ed espressività:

- **ipotesi di indipendenza spesso violata;**

- **sensibilità a dati sparsi e probabilità nulle;**

Il Naive Bayes stima le probabilità condizionate tramite frequenze osservate. Se una certa combinazione attributo–classe non compare nel dataset di training, la probabilità stimata risulta nulla:

$$P(x_i | v) = 0$$

Poiché la regola di decisione prevede il prodotto delle probabilità condizionate, una singola probabilità nulla è sufficiente ad annullare l'intero termine. Questo problema è particolarmente rilevante in dataset ad alta dimensionalità o con pochi esempi.

- **scarsa calibrazione delle probabilità;**

Le prestazioni del Naive Bayes dipendono fortemente dalla rappresentazione degli attributi. Attributi ridondanti o altamente correlati violano l'assunzione di indipendenza e possono distorcere il calcolo delle probabilità.

Inoltre, per attributi continui, l'assunzione di una specifica distribuzione parametrica (ad esempio gaussiana) può risultare inadeguata se i dati reali non seguono tale distribuzione.

- **esistono modelli più espressivi;**

Nonostante ciò, il Naive Bayes rimane un modello di riferimento per problemi di classificazione ad alta dimensionalità e con dati limitati.

RIEPILOGO

Bayesian Learning Approccio probabilistico all'apprendimento che rappresenta esplicitamente l'incertezza. L'obiettivo non è selezionare una singola ipotesi, ma stimare distribuzioni di probabilità aggiornate sulla base dei dati osservati.

Classificazione come stima probabilistica Dato un dataset D e una nuova istanza x' , la classificazione consiste nel calcolare:

$$v^* = \arg \max_{v \in V} P(v \mid x', D)$$

ovvero la classe con massima probabilità a posteriori.

Maximum Likelihood (ML) Assume classi equiprobabili a priori. La decisione si basa esclusivamente sulla verosimiglianza:

$$v_{ML} = \arg \max_{v \in V} P(x' \mid v, D)$$

Maximum A Posteriori (MAP) Estende il ML includendo le probabilità a priori delle classi:

$$v_{MAP} = \arg \max_{v \in V} P(x' \mid v, D) P(v \mid D)$$

Il ML è un caso particolare del MAP con prior uniformi.

Bayes Optimal Classifier Classificatore teoricamente ottimale che combina le predizioni di tutte le ipotesi pesate dalla loro probabilità a posteriori:

$$v^* = \arg \max_{v \in V} \sum_{h \in H} P(v \mid x', h) P(h \mid D)$$

È generalmente intrattabile nella pratica.

Naive Bayes Classifier

Approssimazione pratica del Bayes Optimal Classifier basata sull'assunzione di indipendenza condizionata degli attributi dato il valore della classe:

$$P(x \mid v) = \prod_{i=1}^n P(x_i \mid v)$$

La regola di decisione è:

$$v_{NB} = \arg \max_{v \in V} P(v) \prod_{i=1}^n P(x_i \mid v)$$

Assunzione di indipendenza

Se un'istanza è descritta da $x = (x_1, x_2, \dots, x_n)$, si assume che:

$$P(x_1, x_2, \dots, x_n \mid v) = \prod_{i=1}^n P(x_i \mid v)$$

Questa assunzione implica che, fissata la classe, il valore di ciascun attributo non fornisce informazioni sugli altri attributi.

Perché funziona Nonostante l'ipotesi semplificativa:

- confronta probabilità relative tra classi;
- ha bassa varianza;
- è robusto con pochi dati e in spazi ad alta dimensionalità.

Limiti principali

- ipotesi di indipendenza spesso violata;
- sensibilità a dati sparsi e probabilità nulle;
- scarsa calibrazione delle probabilità;
- limitata capacità espressiva rispetto a modelli più complessi.

Chapter 5

Probabilistic Models for Classification

Nel problema di classificazione supervisionata si assume l'esistenza di una funzione target

$$f : X \rightarrow \mathcal{C},$$

dove $X \subseteq \mathbb{R}^d$ è lo spazio delle istanze e $\mathcal{C} = \{C_1, \dots, C_K\}$ è l'insieme delle classi. Dato un dataset di training $D = \{(x_n, t_n)\}_{n=1}^N$ e una nuova istanza $x \notin D$, l'obiettivo è stimare la probabilità a posteriori

$$P(C_k | x),$$

e assegnare la classe più probabile.

I modelli probabilistici per la classificazione si suddividono in due grandi famiglie: modelli generativi e modelli discriminativi.

5.1 Modelli probabilistici generativi

I modelli generativi stimano esplicitamente la distribuzione dei dati condizionata alla classe $P(x | C_k)$ e la probabilità a priori delle classi $P(C_k)$. La probabilità a posteriori viene poi ottenuta tramite il teorema di Bayes.

Nel caso di due classi C_1, C_2 , assumendo un modello parametrico gaussiano, si ha:

$$P(C_1) = \pi, \quad P(C_2) = 1 - \pi,$$

$$P(x | C_i) = \mathcal{N}(x; \mu_i, \Sigma).$$

La probabilità a posteriori può essere scritta nella forma:

$$P(C_1 | x) = \sigma(a), \quad a = \ln \frac{P(x | C_1)P(C_1)}{P(x | C_2)P(C_2)},$$

dove $\sigma(\cdot)$ è la funzione sigmoide.

5.1.1 Caso gaussiano con covarianza condivisa

Se le due classi condividono la stessa matrice di covarianza Σ , il termine a diventa una funzione lineare dell'input:

$$a = w^T x + w_0,$$

con

$$w = \Sigma^{-1}(\mu_1 - \mu_2), \quad w_0 = -\frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mu_2^T \Sigma^{-1} \mu_2 + \ln \frac{\pi}{1 - \pi}.$$

La classificazione avviene confrontando $P(C_1 | x)$ con una soglia (tipicamente 0.5).

5.1.2 Stima dei parametri tramite Maximum Likelihood

Assumendo un modello gaussiano, i parametri $\pi, \mu_1, \mu_2, \Sigma$ vengono stimati massimizzando la likelihood sul dataset di training. Nel caso binario, le soluzioni ML sono:

$$\hat{\pi} = \frac{N_1}{N}, \quad \hat{\mu}_1 = \frac{1}{N_1} \sum_{n:t_n=1} x_n, \quad \hat{\mu}_2 = \frac{1}{N_2} \sum_{n:t_n=0} x_n, \\ \hat{\Sigma} = \frac{N_1}{N} S_1 + \frac{N_2}{N} S_2,$$

dove S_i sono le matrici di covarianza empiriche delle classi.

5.1.3 Estensione a K classi

Nel caso multi-classe, la probabilità a posteriori assume la forma:

$$P(C_k | x) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}, \quad a_k = \ln P(x | C_k) P(C_k),$$

che corrisponde alla funzione *softmax*.

Anche in questo caso, assumendo distribuzioni gaussiane con covarianza condivisa, i parametri possono essere stimati tramite Maximum Likelihood.

5.1.4 Gaussian Naive Bayes

Nel Gaussian Naive Bayes si assume l'indipendenza condizionata delle feature dato il valore della classe:

$$P(x | C_k) = \prod_{j=1}^m P(x_j | C_k),$$

con

$$P(x_j | C_k) = \mathcal{N}(x_j; \mu_{j,k}, \sigma_{j,k}^2).$$

Questa assunzione riduce drasticamente il numero di parametri da stimare e rende il modello particolarmente adatto a problemi ad alta dimensionalità.

5.2 Modelli probabilistici discriminativi

I modelli discriminativi stimano direttamente la probabilità a posteriori $P(C_k | x)$ senza modellare $P(x | C_k)$. Un esempio fondamentale è la regressione logistica.

5.2.1 Logistic Regression

Nel caso binario, la regressione logistica modella la probabilità come:

$$P(C_1 | \tilde{x}) = \sigma(\tilde{w}^T \tilde{x}),$$

dove $\tilde{x} = (1, x^T)^T$.

I parametri vengono stimati minimizzando la cross-entropy (negative log-likelihood):

$$E(\tilde{w}) = - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)].$$

L'ottimizzazione viene risolta tramite metodi iterativi, in particolare l'algoritmo *Iterative Reweighted Least Squares* (IRLS).

5.2.2 Regressione logistica multi-classe

Per K classi, la probabilità a posteriori è modellata tramite softmax:

$$P(C_k | \tilde{x}) = \frac{\exp(\tilde{w}_k^T \tilde{x})}{\sum_j \exp(\tilde{w}_j^T \tilde{x})}.$$

Anche in questo caso, i parametri sono stimati minimizzando la cross-entropy multi-classe tramite ottimizzazione iterativa.

5.2.3 Generalizzazione e spazio delle feature

Tutti i modelli descritti possono essere applicati in uno spazio delle feature trasformato. Dato un mapping

$$\phi : X \rightarrow \Phi,$$

le istanze vengono proiettate nello spazio Φ e l'apprendimento avviene sulle nuove rappresentazioni.

Questo approccio consente di aumentare la capacità espressiva del modello senza modificarne la struttura probabilistica.

RIEPILOGO

Classificazione probabilistica

Nel problema di classificazione si stima la probabilità a posteriori delle classi dato un input x :

$$P(C_k | x)$$

La decisione consiste nell'assegnare la classe con probabilità massima.

Modelli generativi

Stimano esplicitamente la distribuzione dei dati e delle classi:

$$P(x | C_k), \quad P(C_k)$$

La probabilità a posteriori è ottenuta tramite il teorema di Bayes. Esempi: Gaussian Classifier, Gaussian Naive Bayes.

Classificatore gaussiano

Assumendo distribuzioni gaussiane per ciascuna classe:

$$P(x | C_k) = \mathcal{N}(x; \mu_k, \Sigma_k)$$

Se le classi condividono la stessa covarianza, la funzione discriminante è lineare:

$$a_k = w_k^T x + w_{k0}$$

Stima dei parametri (ML)

I parametri del modello generativo vengono stimati tramite Maximum Likelihood:

$$\hat{\pi}_k = \frac{N_k}{N}, \quad \hat{\mu}_k = \frac{1}{N_k} \sum_{n:t_n=k} x_n$$

Estensione multi-classe

Nel caso di K classi, la probabilità a posteriori assume la forma softmax:

$$P(C_k | x) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$$

Gaussian Naive Bayes

Assume indipendenza condizionata delle feature dato il valore della classe:

$$P(x | C_k) = \prod_{j=1}^m P(x_j | C_k)$$

Riduce drasticamente il numero di parametri ed è adatto a problemi ad alta dimensionalità.

Modelli discriminativi

Stimano direttamente la probabilità a posteriori $P(C_k | x)$, senza modellare la distribuzione dei dati. Esempio fondamentale: regressione logistica.

Logistic Regression

Nel caso binario:

$$P(C_1 | x) = \sigma(w^T x)$$

I parametri sono stimati minimizzando la cross-entropy tramite ottimizzazione iterativa (IRLS).

Logistic Regression multi-classe

Utilizza la funzione softmax:

$$P(C_k | x) = \frac{\exp(w_k^T x)}{\sum_j \exp(w_j^T x)}$$

Spazio delle feature

I modelli probabilistici possono operare in uno spazio trasformato tramite una mappa:

$$\phi : X \rightarrow \Phi$$

Aumentando la capacità espressiva senza modificare la struttura del modello.

Chapter 6

Modelli lineari per la classificazione

6.1 Dati linearmente separabili

Un insieme di dati è detto **linearmente separabile** se esiste un iperpiano che separa perfettamente le istanze appartenenti a classi diverse.

Formalmente, dati esempi $x \in \mathbb{R}^d$ appartenenti a due classi, essi sono linearmente separabili se esistono un vettore $w \in \mathbb{R}^d$ e uno scalare $w_0 \in \mathbb{R}$ tali che:

$$w^T x + w_0 > 0 \quad \text{per tutti gli esempi della prima classe}$$

$$w^T x + w_0 < 0 \quad \text{per tutti gli esempi della seconda classe}$$

In questo caso, l'iperpiano definito dall'equazione $w^T x + w_0 = 0$ separa esattamente le due classi.

Interpretazione geometrica

L'iperpiano $w^T x + w_0 = 0$ divide lo spazio delle feature in due semispazi. Il segno della funzione discriminante determina la classe assegnata all'istanza.

Nel caso bidimensionale, l'iperpiano corrisponde a una retta; nel caso tridimensionale, a un piano. In dimensioni superiori, la separazione avviene tramite un iperpiano.

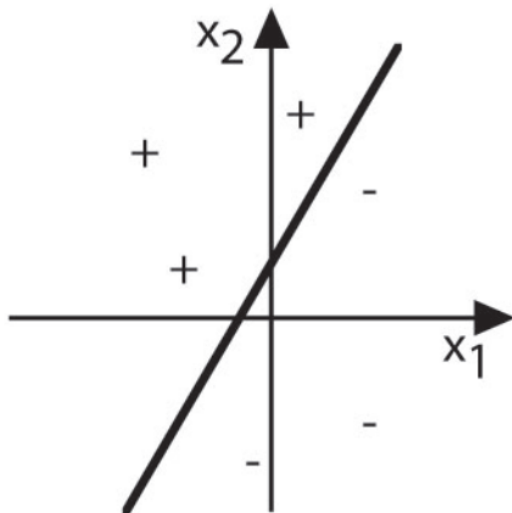


Figure 6.1: Esempio di dati linearmente separabili tramite una retta.

6.1.1 Dati non linearmente separabili

Un insieme di dati è detto **non linearmente separabile** se non esiste alcun iperpiano in grado di separare perfettamente le classi.

In tali casi, una funzione discriminante lineare non è sufficiente a risolvere il problema di classificazione, rendendo necessaria l'introduzione di modelli non lineari o di trasformazioni dello spazio delle feature.

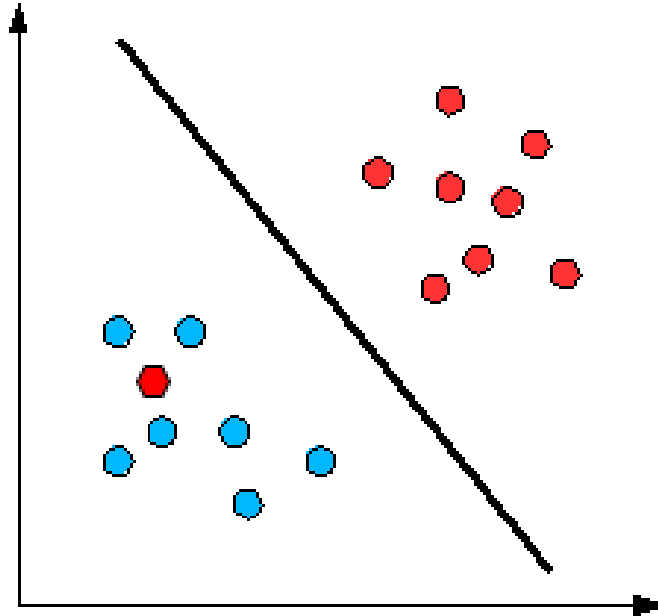


Figure 6.2: Esempio di dati non linearmente separabili.

6.2 Linear Discriminant Function

Una **funzione discriminante lineare** è una funzione del tipo:

$$g(x) = w^T x + w_0$$

dove:

- $w \in \mathbb{R}^d$ è il vettore dei pesi;
- $w_0 \in \mathbb{R}$ è il termine di bias;
- $g(x) \in \mathbb{R}$ è il valore della funzione discriminante.

Essa associa a ogni istanza un valore reale, il cui segno viene utilizzato per effettuare la classificazione.

6.2.1 Regola di decisione

La regola di classificazione basata su una funzione discriminante lineare è:

$$\text{assegna } x \text{ alla classe } \begin{cases} C_1 & \text{se } g(x) \geq 0 \\ C_2 & \text{se } g(x) < 0 \end{cases}$$

Il valore assoluto di $g(x)$ fornisce una misura della distanza dell'istanza dall'iperpiano di separazione.

Interpretazione geometrica

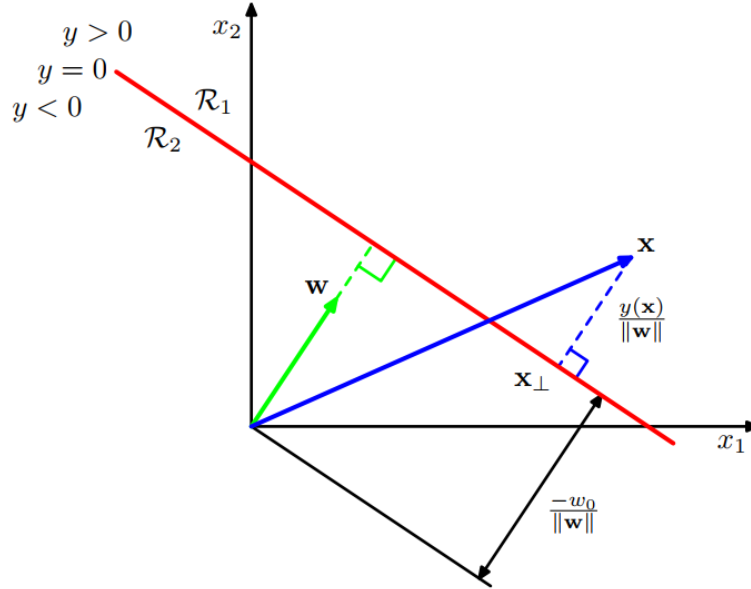


Figure 6.3: Interpretazione geometrica di una funzione discriminante lineare in due dimensioni.

Il vettore dei pesi w è ortogonale all'iperpiano di decisione. Esso indica la direzione di massima crescita della funzione discriminante.

Nella figura, il vettore w è rappresentato come una freccia perpendicolare alla retta di separazione.

Dato un punto x , il valore della funzione discriminante $g(x)$ è proporzionale alla distanza con segno del punto dall'iperpiano.

La distanza geometrica di x dall'iperpiano è data da:

$$\frac{g(x)}{\|w\|}$$

Nella figura, $x_{\perp}$ rappresenta la proiezione ortogonale del punto x sull'iperpiano.

Il termine di bias w_0 determina la posizione dell'iperpiano rispetto all'origine. In particolare, la distanza dell'iperpiano dall'origine è:

$$-\frac{w_0}{\|w\|}$$

Questo valore controlla la traslazione dell'iperpiano senza modificarne l'orientamento.

6.3 Learning di funzioni discriminanti lineari

Nei paragrafi precedenti è stata introdotta la funzione discriminante lineare come strumento per separare due classi mediante un iperpiano nello spazio delle feature. Resta ora da affrontare il problema fondamentale dell'apprendimento: come determinare i parametri w e w_0 a partire da un insieme di dati etichettati.

I **Linear Discriminant Learning Algorithms** comprendono una famiglia di metodi che apprendono una funzione discriminante lineare a partire da esempi di training. Tali algoritmi condividono l'obiettivo di costruire una funzione del tipo

$$g(x) = w^T x + w_0$$

in grado di separare, nel miglior modo possibile, le istanze appartenenti a classi diverse.

Esistono diversi criteri secondo cui è possibile apprendere una funzione discriminante lineare. Alcuni algoritmi mirano a separare correttamente i dati, altri a massimizzare una misura di separazione tra le classi, altri ancora a minimizzare un errore globale.

Tra i principali approcci rientrano:

- il **criterio dei minimi quadrati**, che apprende i parametri minimizzando l'errore quadratico tra output desiderato e output del modello;
- il **Perceptron**, che aggiorna iterativamente i pesi correggendo gli errori di classificazione;
- il **Fisher Linear Discriminant**, che massimizza la separazione statistica tra le classi.
- il **Support Vector Machine**, che trova l'iperpiano ottimale che separa i dati in classi diverse massimizzando il margine tra esse.

Nonostante le differenze nei criteri di ottimizzazione, tutti questi metodi producono una funzione discriminante lineare.

6.4 Least Squares per la classificazione

Il criterio dei **minimi quadrati** rappresenta uno dei metodi più semplici e fondamentali per apprendere una funzione discriminante lineare a partire da un insieme di dati etichettati. L'idea di base consiste nel trattare il problema di classificazione come un problema di approssimazione funzionale.

6.4.1 Definizione del problema

Si consideri un dataset di training:

$$\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N$$

dove:

- $x_n \in \mathbb{R}^d$ è il vettore delle feature dell' n -esimo esempio;
- $t_n \in \mathbb{R}$ è il valore target associato all'esempio.

Nel contesto della classificazione binaria, le etichette sono tipicamente codificate come:

$$t_n \in \{-1, +1\}$$

Si assume un modello lineare della forma:

$$y(x) = w^T x + w_0$$

Introducendo il vettore delle feature esteso:

$$\tilde{x} = \begin{pmatrix} 1 \\ x \end{pmatrix}, \quad \tilde{w} = \begin{pmatrix} w_0 \\ w \end{pmatrix},$$

il modello può essere riscritto in forma compatta come:

$$y(x) = \tilde{w}^T \tilde{x}$$

6.4.2 Funzione di costo

Il criterio dei minimi quadrati consiste nel minimizzare la somma degli errori quadratici tra l'output del modello e i target desiderati:

$$E(\tilde{w}) = \frac{1}{2} \sum_{n=1}^N (y(x_n) - t_n)^2$$

La funzione di costo misura quanto il modello si discosta dai valori target desiderati sull'insieme di training. Per ciascun esempio x_n , il termine

$$(y(x_n) - t_n)^2$$

rappresenta l'errore commesso dal modello su quell'esempio.

La somma degli errori quadratici tiene conto di tutti gli esempi del dataset e penalizza maggiormente gli errori di grande entità. Minimizzare la funzione di costo equivale quindi a trovare i parametri w e w_0 che rendono l'output del modello il più vicino possibile ai target, in media, su tutti i dati di training.

6.4.3 Soluzione ottima

Il minimo della funzione di costo si ottiene imponendo l'annullamento del gradiente:

$$\nabla E(\tilde{w}) = 0$$

Da cui si ricavano le **equazioni normali**:

$$\tilde{X}^T \tilde{X} \tilde{w} = \tilde{X}^T t$$

Se la matrice $\tilde{X}^T \tilde{X}$ è invertibile, la soluzione è:

$$\tilde{w} = (\tilde{X}^T \tilde{X})^{-1} \tilde{X}^T t$$

La soluzione ottima del problema dei minimi quadrati è il vettore dei parametri che minimizza globalmente la funzione di costo. Poiché la funzione di costo è quadratica nei parametri, essa è convessa e ammette un unico minimo globale.

Questo significa che la soluzione ottenuta risolvendo le equazioni normali è la migliore possibile secondo il criterio dei minimi quadrati, e non dipende dall'inizializzazione dei parametri.

6.4.4 Regola di classificazione

Una volta stimati i parametri $\tilde{w}$, la classificazione di una nuova istanza x avviene utilizzando il segno dell'output del modello:

$$\hat{t} = \text{sign}(y(x)) = \text{sign}(w^T x + w_0)$$

Il valore continuo $y(x)$ può essere interpretato come una misura di confidenza rispetto alla decisione di classificazione.

6.4.5 Sensibilità agli outlier

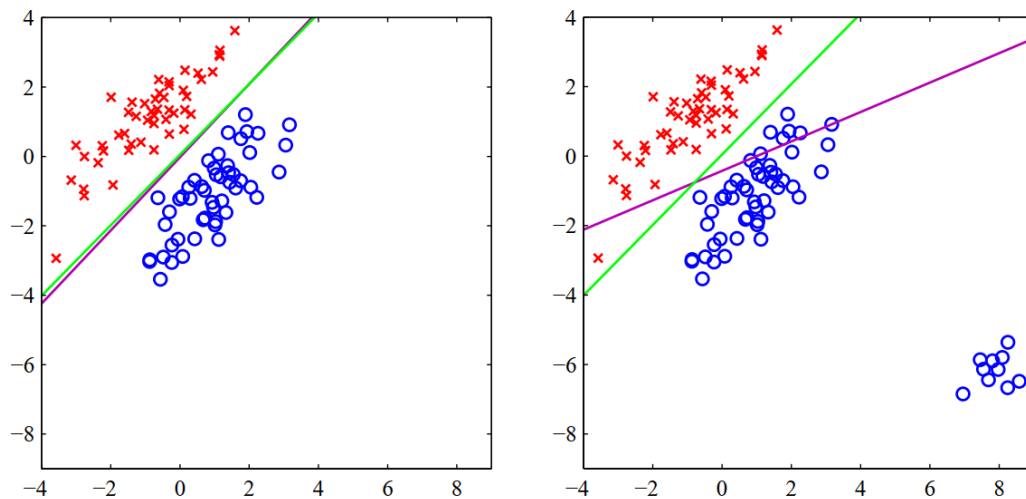


Figure 6.4: Effetto degli outlier sul metodo dei minimi quadrati. L'aggiunta di pochi punti anomali può modificare significativamente la frontiera di decisione.

La figura illustra l'effetto degli outlier sull'apprendimento di una funzione discriminante lineare tramite il criterio dei minimi quadrati.

Nel pannello di sinistra, le due classi sono ben separate e la funzione discriminante appresa (retta verde) fornisce una separazione ragionevole dei dati. Nel pannello di destra, l'introduzione di pochi punti anomali (outlier) appartenenti alla classe blu, ma lontani dalla distribuzione principale, provoca una variazione significativa della retta di separazione (retta viola).

Questo comportamento è una conseguenza diretta della funzione di costo dei minimi quadrati: Poiché l'errore è elevato al quadrato, gli esempi per cui la differenza tra output del modello e target è grande contribuiscono in modo sproporzionato al valore complessivo della funzione di costo. Di conseguenza, pochi outlier possono dominare l'ottimizzazione e influenzare fortemente la posizione dell'iperpiano appreso.

NOTA BENE:

Un metodo di apprendimento si dice **non robusto agli outlier** quando la presenza di pochi esempi anomali è sufficiente a modificare in modo significativo il modello appreso.

Nel caso del Least Squares, l'algoritmo cerca di ridurre l'errore quadratico complessivo, anche a costo di peggiorare la classificazione della maggioranza dei dati, pur di adattarsi ai punti anomali. Questo limite rende il criterio dei minimi quadrati poco adatto ai problemi di classificazione, in cui l'obiettivo principale è la corretta separazione delle classi piuttosto che l'approssimazione numerica dei target.

Tale osservazione motiva l'introduzione di algoritmi alternativi, come il perceptron o i metodi basati su funzioni di costo non quadratiche, più robusti alla presenza di outlier.

6.5 Perceptron

Il **Perceptron** è uno dei più semplici algoritmi di apprendimento supervisionato per la classificazione binaria. Esso apprende una funzione discriminante lineare a partire da esempi etichettati, aggiornando iterativamente i parametri del modello in base agli errori di classificazione.

Il Perceptron può essere interpretato sia come un algoritmo di apprendimento per funzioni discriminanti lineari, sia come un modello elementare di neurone artificiale.

6.5.1 Definizione del problema

Si consideri un insieme di dati di training:

$$\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N$$

dove:

- $x_n \in \mathbb{R}^d$ è il vettore delle feature dell' n -esimo esempio;
- $t_n \in \{-1, +1\}$ è l'etichetta di classe associata.

L'obiettivo è apprendere una funzione:

$$f : \mathbb{R}^d \rightarrow \{-1, +1\}$$

che assegni correttamente un'etichetta di classe a nuove istanze non osservate.

6.5.2 Modello del Perceptron

Il Perceptron utilizza un modello lineare della forma:

$$g(x) = w^T x + w_0$$

L'output del classificatore è ottenuto applicando una funzione di soglia:

$$o(x) = \begin{cases} +1 & \text{se } g(x) > 0 \\ -1 & \text{altrimenti} \end{cases} = \text{sign}(w^T x + w_0)$$

La funzione di soglia separa lo spazio delle feature in due regioni distinte, corrispondenti alle due classi. Il valore assoluto di $g(x)$ indica quanto il punto si trovi lontano dall'iperpiano di decisione, ma non influisce direttamente sulla decisione finale.

Questo comportamento distingue il Perceptron da modelli probabilistici, nei quali l'output rappresenta una probabilità.

6.5.3 Perceptron Learning Rule

L'apprendimento nel Perceptron avviene in modo iterativo, aggiornando i pesi solo quando un esempio di training viene classificato in modo errato.

Dato un esempio (x_n, t_n) , con $t_n \in \{-1, +1\}$, il modello produce un output:

$$o(x_n) = \text{sign}(w^T x_n + w_0).$$

Se $o(x_n) = t_n$, l'esempio è classificato correttamente e non viene effettuato alcun aggiornamento. Se invece $o(x_n) \neq t_n$, i parametri vengono aggiornati secondo la seguente regola.

In presenza di errore di classificazione, i pesi vengono aggiornati come:

$$w \leftarrow w + \eta t_n x_n$$

$$w_0 \leftarrow w_0 + \eta t_n$$

dove $\eta > 0$ è il tasso di apprendimento.

L'aggiornamento dei pesi modifica l'iperpiano di decisione in modo da ridurre l'errore commesso sull'esempio corrente.

In particolare:

- se un esempio positivo viene classificato come negativo, il termine $t_n x_n$ spinge l'iperpiano verso l'esempio;
- se un esempio negativo viene classificato come positivo, l'aggiornamento avviene nella direzione opposta.

In questo modo, l'iperpiano viene progressivamente adattato per separare correttamente i dati.

Forma compatta

La regola di aggiornamento del Perceptron può essere scritta in forma compatta come:

$$w \leftarrow w + \eta t_n x_n \quad \text{se } t_n(w^T x_n + w_0) \leq 0.$$

Questa espressione evidenzia che l'aggiornamento avviene esclusivamente in presenza di errore.

6.5.4 Algoritmo del Perceptron

Input:

Training set $D = \{(x_n, t_n)\}$, $t_n \in \{-1, +1\}$
Learning rate $\eta > 0$

Initialize:

$w = 0$
 $w_0 = 0$

Repeat until convergence:

```
for each  $(x_n, t_n)$  in  $D$ :  
    if  $t_n * (w^T x_n + w_0) \leq 0$  then  
         $w \leftarrow w + \eta * t_n * x_n$   
         $w_0 \leftarrow w_0 + \eta * t_n$   
    end if  
end for
```

L'algoritmo del Perceptron procede iterativamente scorrendo il dataset di training. Per ciascun esempio, viene verificata la correttezza della classificazione.

Se un esempio è classificato correttamente, i parametri restano invariati. In caso contrario, i pesi e il bias vengono aggiornati per correggere l'errore. L'algoritmo termina quando un intero passaggio sui dati non produce aggiornamenti.

Esempio di applicazione

Si consideri un semplice dataset bidimensionale con target binari:

$$\mathcal{D} = \{(x_1, t_1), (x_2, t_2), (x_3, t_3)\}$$

con:

$$\begin{aligned}x_1 &= (1, 1), & t_1 &= +1 \\x_2 &= (2, 0), & t_2 &= +1 \\x_3 &= (-1, -1), & t_3 &= -1\end{aligned}$$

Si inizializzano i parametri:

$$w = (0, 0), \quad w_0 = 0$$

e si assume $\eta = 1$.

Per il primo esempio $x_1 = (1, 1)$:

$$g(x_1) = w^T x_1 + w_0 = 0$$

Poiché:

$$t_1 g(x_1) = 0 \leq 0,$$

l'esempio è considerato errato e si applica l'aggiornamento:

$$w \leftarrow (0, 0) + 1 \cdot (+1) \cdot (1, 1) = (1, 1)$$

$$w_0 \leftarrow 0 + 1 \cdot (+1) = 1$$

Per $x_2 = (2, 0)$:

$$g(x_2) = (1, 1)^T(2, 0) + 1 = 3$$

Poiché:

$$t_2 g(x_2) = (+1) \cdot 3 > 0,$$

l'esempio è classificato correttamente e non viene effettuato alcun aggiornamento.

Per $x_3 = (-1, -1)$:

$$g(x_3) = (1, 1)^T(-1, -1) + 1 = -1$$

Poiché:

$$t_3 g(x_3) = (-1) \cdot (-1) > 0,$$

anche questo esempio è classificato correttamente.

Al termine del primo ciclo sui dati, non si verificano ulteriori errori. L'iperpiano appreso è quindi:

$$w^T x + w_0 = 0 \quad \Rightarrow \quad x_1 + x_2 + 1 = 0$$

Questo iperpiano separa correttamente le due classi del dataset.

Visualizzazione dell'algoritmo del Perceptron

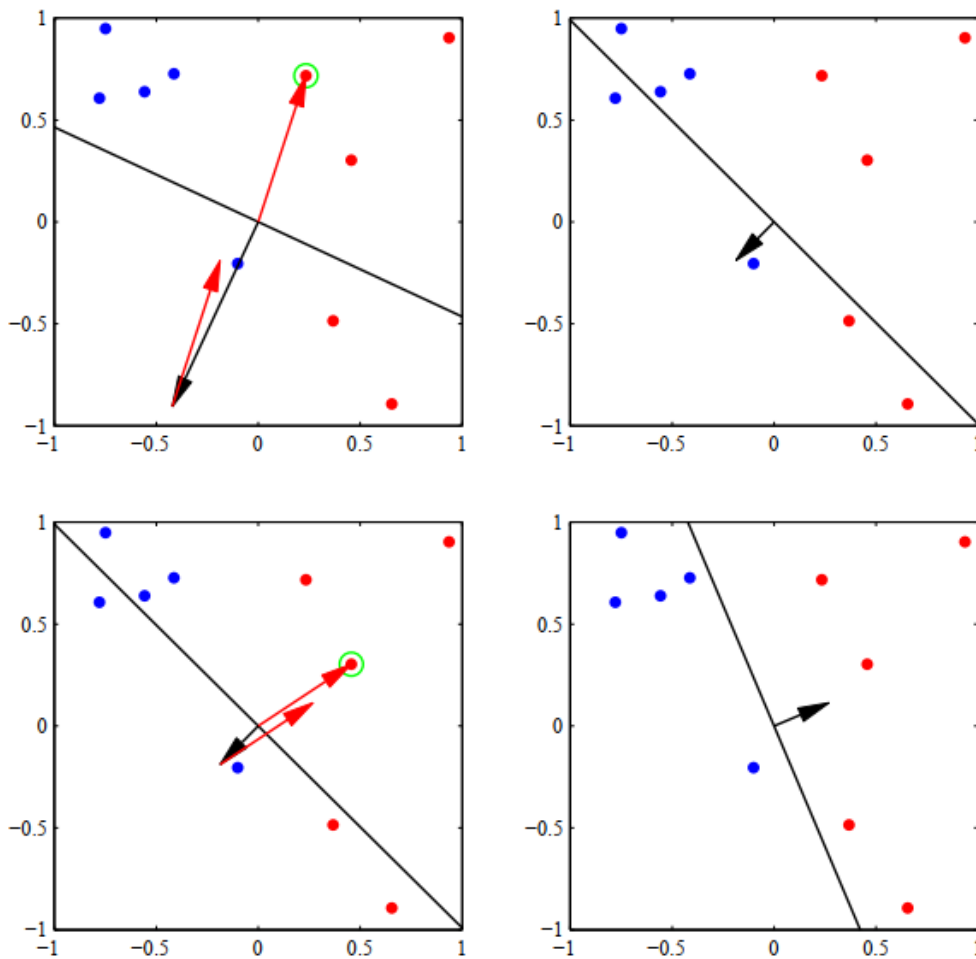


Figure 6.5: Evoluzione dell'iperpiano di decisione durante l'apprendimento del Perceptron. Ogni pannello rappresenta un aggiornamento successivo dei pesi in risposta a un errore di classificazione.

La figura mostra il comportamento dell'algoritmo del Perceptron durante il processo di apprendimento su un dataset bidimensionale.

In ciascun pannello è rappresentato l'iperpiano di decisione corrente, insieme agli esempi di training appartenenti alle due classi. Le frecce indicano la direzione dell'aggiornamento dei pesi quando viene incontrato un esempio classificato erroneamente. Quando un esempio viene classificato in modo errato, il Perceptron modifica i parametri del modello secondo la regola:

$$w \leftarrow w + \eta t_n x_n$$

Questo aggiornamento sposta l'iperpiano nella direzione dell'esempio mal classificato se $t_n = +1$, o nella direzione opposta se $t_n = -1$. Nella figura, i punti cerchiati evidenziano gli esempi che causano l'aggiornamento dei pesi.

Dal punto di vista geometrico, ogni aggiornamento ruota e trasla l'iperpiano di decisione nel tentativo di ridurre il numero di errori di classificazione.

Il Perceptron non ottimizza una funzione di costo globale, ma corregge localmente l'errore commesso sull'esempio corrente. Di conseguenza, la traiettoria seguita dall'iperpiano dipende dall'ordine con cui vengono presentati i dati di training.

6.5.5 Interpretazione neurale

Il perceptron è la forma più semplice di rete neurale: consiste in un singolo neurone artificiale che riceve input pesati, li somma, applica una funzione di attivazione (tipicamente a gradino) e produce un output binario per classificare linearmente i dati. Storicamente è stato il primo modello di rete neurale (proposto da Rosenblatt nel 1958) e ha posto le basi concettuali per le reti neurali moderne. La sua limitazione principale è che può risolvere solo problemi linearmente separabili - per questo motivo le reti neurali si sono evolute in architetture multi-strato (multilayer perceptron o MLP) capaci di apprendere relazioni non lineari grazie ai neuroni nascosti e alla retropropagazione dell'errore.

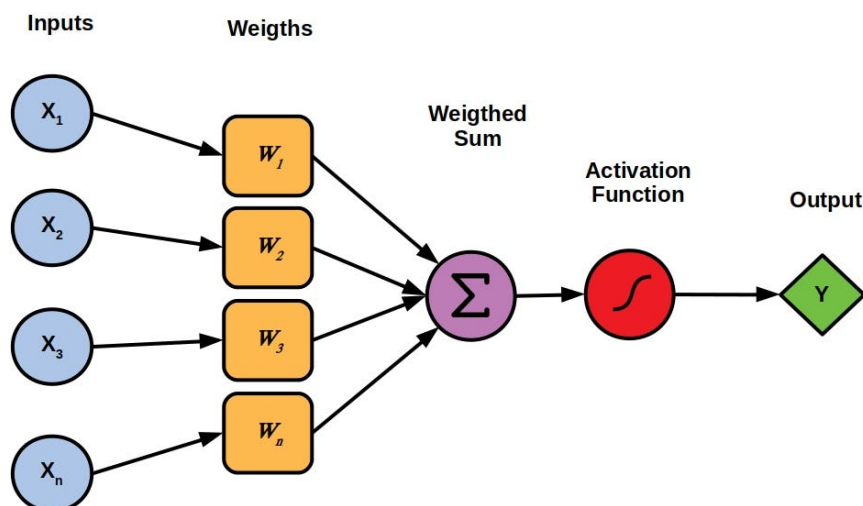


Figure 6.6: Visualizzazione grafica del Perceptron come neurone artificiale.

Nell'immagine vediamo come ciascun input x_i è associato a un peso w_i , mentre il termine di bias è modellato tramite un input costante $x_0 = 1$ con peso w_0 .

Il neurone calcola una combinazione lineare degli input:

$$\sum_{i=0}^d w_i x_i$$

e applica una funzione di attivazione a soglia che restituisce l'etichetta di classe.